STUDY NOTES · VIDEO 1 OF 5

What is Claude & Claude Code?

Anthropic · LLMs · Model family · Claude vs Claude Code · Agentic Loop

```
PS C:\Users\you> claude
✓ Same brain. Now with hands.
```

Notes written for a Windows user · AI-engineer perspective added

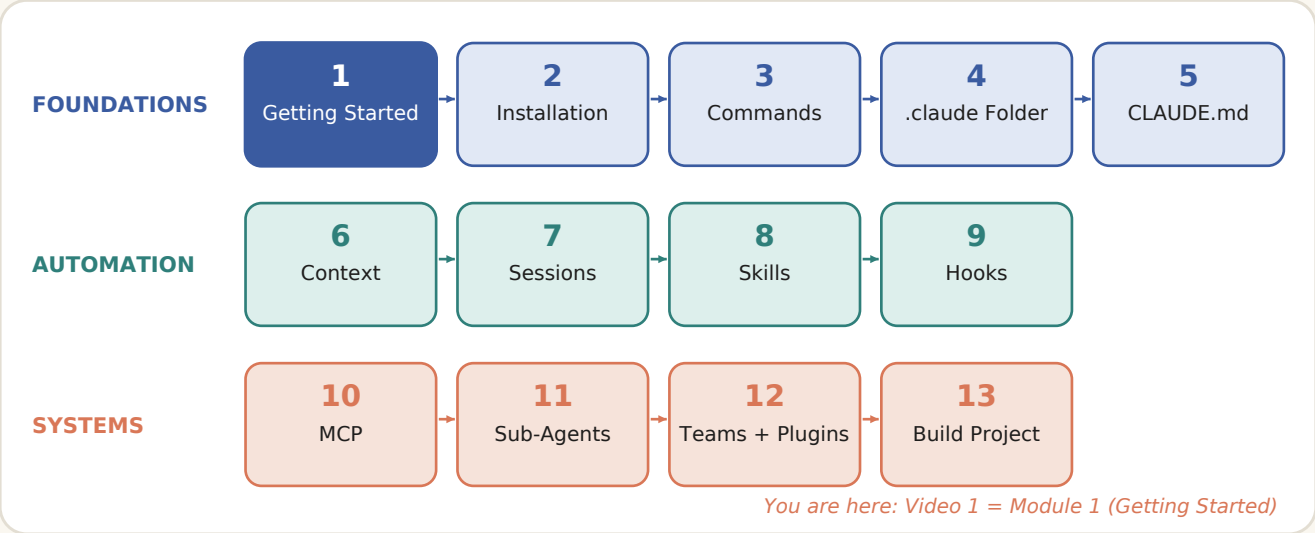
What's inside

01	The big picture — course roadmap
02	Anthropic & Claude (vs OpenAI & ChatGPT)
03	LLM = the brain (next-token prediction)
04	The Claude model family: Haiku · Sonnet · Opus
05	Claude on Web & Windows Desktop
06	The Terminal — where 'hands' come from
07	Claude Chat vs Claude Code
08	The Agentic Loop (most important idea)
09	AI Engineer corner — projects, use-cases, interview Qs
10	Cheat sheet, resources & next video

SECTION 01

The big picture

Before installing anything, understand the ecosystem. Fundamentals first — so the same knowledge works for GitHub Copilot, Codex or any other coding agent too.



Course roadmap (13 modules) — based on the creator's GitHub mindmap

- **Foundations (1–5):** what Claude Code is, installation, commands & permission modes, the `.claude` folder, `CLAUDE.md` (project memory).
- **Automation (6–9):** context window & tokens, sessions & `/compact`, Skills (reusable workflows), Hooks (auto scripts / guardrails).
- **Systems (10–13):** MCP (external tools: GitHub, APIs, Gmail), Sub-agents, Teams + Plugins, and a final end-to-end project.

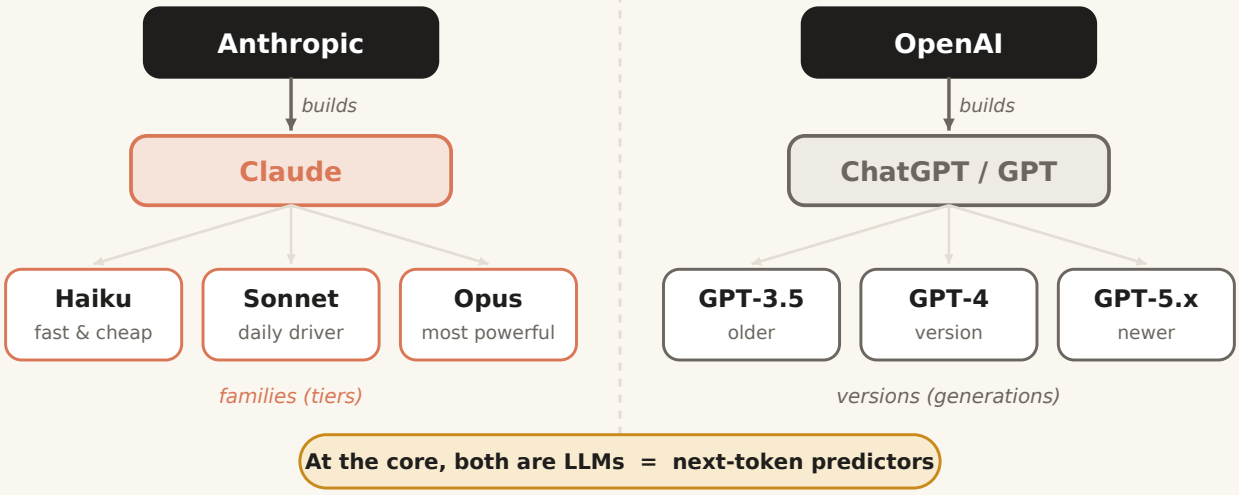
> ★ **Yaad rakho**
> Log jaldi install karke frustrate hote hain kyun ke fundamentals clear nahi hote. Is video ka goal: **samajhna ke Claude Code kya hai aur kyun alag hai** — install agli video mein.

SECTION 02

Anthropic & Claude

Who builds Claude, and how it maps onto what you already know from ChatGPT.

	OpenAI side	Anthropic side
Company	OpenAI (Sam Altman)	Anthropic (anthropic.com)
Product	ChatGPT	Claude (claude.ai)
Model naming	Versions: GPT-3.5 → GPT-4 → GPT-5.x	Families/tiers first: Haiku · Sonnet · Opus — each with versions
Philosophy	Made AI mainstream ('Google of AI')	Founded by ex-OpenAI people; **safety-first** — powerful *and* trustworthy



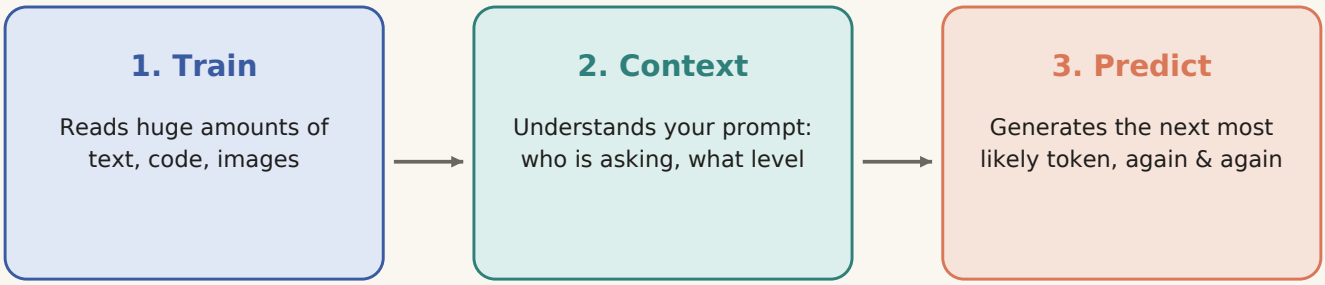
Same idea, different philosophy: Anthropic names tiers, OpenAI names generations

★ **Yaad rakho**
OpenAI → ChatGPT, waise hi **Anthropic → Claude**. Agar ChatGPT samajh aata hai to Claude bhi seedha 'parallel' karke samajh lo.

SECTION 03

LLM = the brain

Claude, ChatGPT and Gemini are all Large Language Models at their core.



`Plants` `make` `their` `own` `food` `...` *<- one token at a time (that's why text 'streams')*

- **Trained** on vast data (text, code, images) — like a person who has read millions of books.
- **Next-token predictor:** it writes the answer one piece at a time. That is why replies appear word-by-word.
- **Context-aware:** the same question gives different answers depending on who is asking.

The demo from the video

Prompt	What Claude did
Explain photosynthesis like I'm 10 (under 500 words)	Simple words, food analogy, no formulas
Explain photosynthesis like I'm a PhD (under 500 words)	Technical language, chemical equation, deeper mechanisms

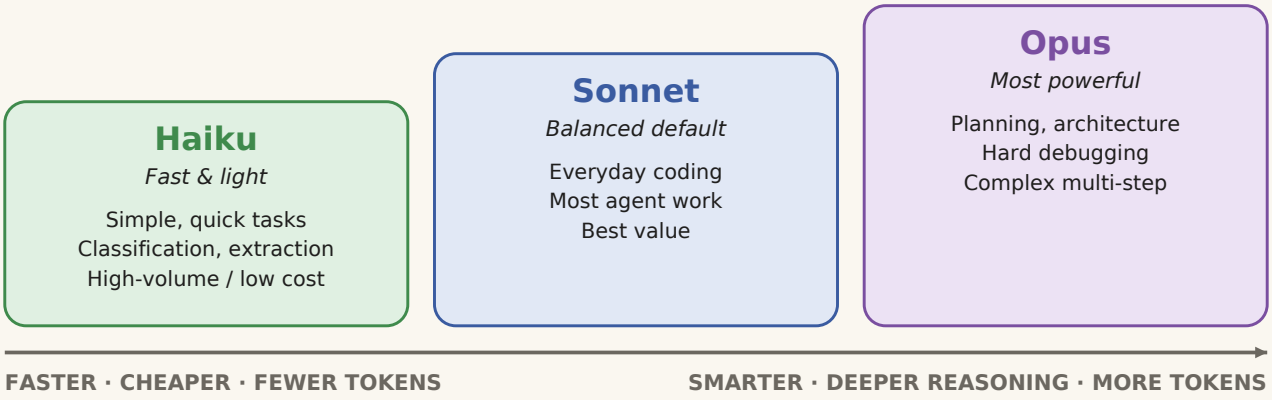
Same question → completely different answer, because the model read the **context** and adapted. Constraints like “under 500 words” also steer it.

> ⚙ **AI Engineer lens**
> A token is roughly **¾ of a word**. Everything is priced, rate-limited and capped by tokens (input + output). The **context window** is the model's working memory for one conversation — when it fills up, quality drops. This becomes critical later (module 6: Context, `/compact`).

SECTION 04

The Claude model family

Speed vs intelligence — pick the right brain for the job.



	Haiku	Sonnet	Opus
Role	Fast, lightweight	Balanced everyday model	Most capable
Use when	Task is simple & quick	Most coding & reasoning	Planning, architecture, hard problems
Token cost	Lowest	Medium	Highest — limits hit faster
Shown in video	Haiku 4.5	Sonnet 4.6	Opus 4.7

Extended / Adaptive thinking

A toggle that lets the model **think longer before answering** — helpful for complex problems and debugging. Flow: *Prompt → Model → (optional) extended thinking → clearer answer*. More thinking = more tokens.

→ **Thumb rule from the video**
Sonnet for everything by default · **Haiku** when the task is very simple and quick · **Opus** when it's deep/complex (planning, architecture). The same choice exists inside Claude Code.

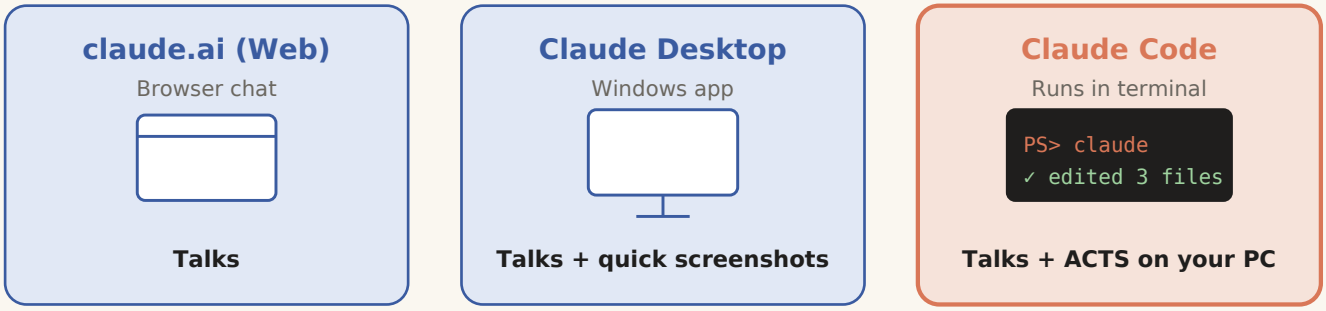
i **Version note**
Model numbers change fast; the family logic doesn't. As of Oct 2026 newer versions exist (e.g. Opus 5.5, Sonnet 5.5, Haiku 5.5). Always check the official **Models overview** page in the docs before choosing.

⚙ **AI Engineer lens**
This is **model routing** — a real production pattern. In your own apps: Haiku for classification/extraction/routing, Sonnet for the main agent, Opus for planning or a final 'hard' pass. Picking the cheapest model that passes your evals saves real money.

SECTION 05

Claude on Web & Windows Desktop

Two ways to chat with the same brain.



Same Claude brain in all three — only the 'hands' change

- **claude.ai (web):** sign up with email/Google, chat like ChatGPT. UI may move around over time — the fundamentals stay.
- **Claude Desktop for Windows:** free download, works like an app (think WhatsApp Web vs WhatsApp app). Extra convenience: quick screenshot/screen sharing from the app.
- **Key limit:** by default, web/desktop Claude **cannot act on your machine** — it can't create your folders or run your files.

★ **Yaad rakho**
Desktop Claude = **phone pe ek smart dost**. Wo bata sakta hai kya karna hai, lekin khud tumhare computer pe kuch nahi kar sakta.

SECTION 06

The Terminal — where the 'hands' come from

Your PC has a GUI (File Explorer), but underneath is the terminal. Whoever can type commands there can control the machine.

On Windows you'll use **PowerShell** (or Windows Terminal running PowerShell). The video showed Mac commands — here are the Windows equivalents:

Goal	PowerShell command	What it does
See where you are	pwd	Prints current folder path
List folder contents	ls (or dir)	Shows files & folders
Go to Desktop	cd $HOME\Desktop	Change directory
Make a folder	mkdir test-claude-code	Creates a new folder
Enter it	cd test-claude-code	Move into the folder
Create empty file	New-Item myfile.txt	Windows version of Mac's touch
Clear screen	cls (or clear)	Cleans the terminal
Run Python file	python sortCode.py	Executes the script

TRY IT IN POWERSHELL

```
PS C:\Users\you> cd $HOME\Desktop
PS C:\Users\you\Desktop> mkdir test-claude-code
PS C:\Users\you\Desktop> cd test-claude-code
PS C:\Users\you\Desktop\test-claude-code> New-Item myfile.txt
PS C:\Users\you\Desktop\test-claude-code> ls      # myfile.txt is there
```

Open File Explorer → Desktop: the folder and file appear — exactly as if you'd right-clicked *New → Folder*. Folder names with spaces need quotes: `mkdir "test folder"`.

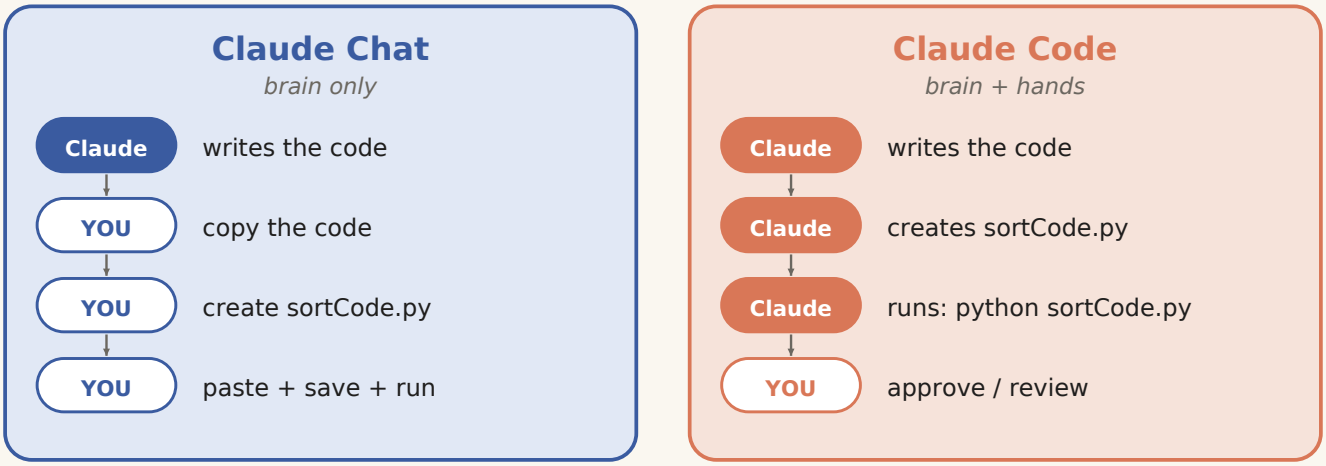
★ **Yaad rakho**

Agar ek **smart brain** ko terminal ka access mil jaye, to wo folder bana sakta hai, file likh sakta hai, code chala sakta hai. **Yahi Claude Code hai** — Claude, jo tumhare terminal mein commands chalata hai.

SECTION 07

Claude Chat vs Claude Code

Same brain, completely different capability. Claude Code is not 'Claude with a new UI' — it's a different category of tool.



Example: "Write a Python bubble sort"

	Claude Chat (web / desktop)	Claude Code
Analogy	A brain	A brain **with hands**
Where	Browser / app	Installed on your PC, runs in the terminal
Output	Text & code to copy	Real files created, edited, run
Who does the work	You do the actions	Claude does them; you approve
Safety	Nothing touches your PC	Asks permission for risky actions

sortCode.py — the demo file (from the course repo: TestClaudeCode/)

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

numbers = [64, 34, 25, 12, 22, 11, 90]
print("Before:", numbers)
print("After: ", bubble_sort(numbers))
```

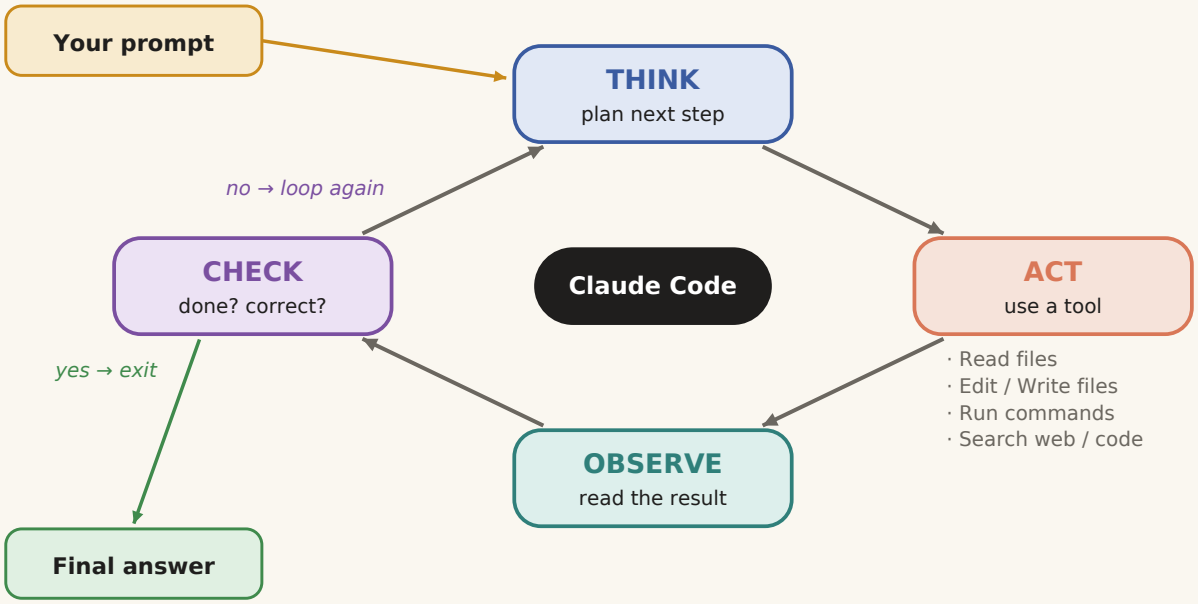
! **Why it needs installing**

To run commands on your PC, Claude Code must live on your PC. That's why it's 'for developers' and why it asks for approval before doing things that matter. Claude's Cowork features work on the same idea.

SECTION 08

The Agentic Loop

The single most important idea in this video. Skills, Hooks, Agents and MCP all plug into this loop.



Think → Act → Observe → Check → repeat until done

- **Think:** understand the goal and plan the next step.
- **Act:** call a tool — read a file, run a command, edit code, search.
- **Observe:** look at what came back (output, errors, file contents).
- **Check:** is it done and correct? If not, loop again. If yes, give the final answer (and ask if you want changes).

The video's live example

Asked Claude to “make a small PDF on bubble sort”. Instead of just answering, it: thought about the request → read its PDF skill → decided to use ReportLab → ran `pip install` → wrote a Python script → ran it → checked the PDF → returned it. **Not magic — fast iteration.** (These very notes were made the same way.)

★ **Yaad rakho**

Tum AI se 'baat' nahi kar rahe, tum ek brain ko **direct** kar rahe ho. Loop = soch → kaam → nateeja dekh → check → dobara.

SECTION 09

AI Engineer corner

How Video 1's concepts map to the work you'll actually do.

The agentic loop, in code

Every agent framework (Claude Code, Agent SDK, LangGraph, your own) is this loop. Understand it once, recognise it everywhere:

PSEUDO-CODE — tool-use agent loop

```
messages = [{"role": "user", "content": task}]
while True:
    resp = llm(model="sonnet", messages=messages, tools=TOOLS)   # THINK
    messages.append(assistant_turn(resp))
    if resp.stop_reason != "tool_use":          # CHECK: no more tools needed
        return resp.text                          # final answer
    for call in resp.tool_calls:                   # ACT
        result = run_tool(call.name, call.input)   # e.g. read_file, bash
        messages.append(tool_result(call.id, result))   # OBSERVE
```

Where Claude Code helps an AI engineer

Task	What you'd ask Claude Code
Scaffold a RAG app	“Create a FastAPI app with /ingest and /query, chunk PDFs, store embeddings, add a README.”
Write evals	“Create a pytest eval set of 30 Q&A; pairs for my RAG and a script that scores accuracy.”
Debug a pipeline	“Run train.py, read the traceback, fix the shape mismatch, rerun until it passes.”
Refactor / tests	“Add type hints and unit tests for src/agents/, keep behaviour identical.”
Explore a repo	“Explain this codebase's architecture and where the prompt templates live.”
Data work	“Clean data.csv, report nulls, save a plot of label distribution.”

Concepts to connect

- **Tool use / function calling** = the 'hands'. Claude Code's built-in tools (read, edit, bash, search) are just tools in the loop.
- **Permissions** = guardrails on actions. In your own agents, gate destructive tools (delete, deploy, payments) behind approval.
- **Tokens & context** = cost and memory. Long loops burn tokens; that's why sub-agents and `/compact` exist (later modules).
- **Model routing** = Haiku / Sonnet / Opus by difficulty (section 04).
- **MCP** (module 10) = an open standard by Anthropic to plug external tools/data into the loop.

Interview-ready questions

Question	Crisp answer
What's the difference between Claude and Claude Code?	Same model; Claude Code runs locally with tools so it can read/edit files and execute commands in an agentic loop, with permission prompts.
What is an agentic loop?	Repeated think → act (tool call) → observe → check until the goal is met, instead of one-shot text generation.
How do you choose a model?	Cheapest model that passes your evals; Haiku for simple/high-volume, Sonnet default, Opus for hard reasoning/planning.
What is an LLM fundamentally?	A model trained on large data that predicts the next token given context.
Why is context window important?	It's the model's working memory; overfilling it raises cost and degrades quality.

SECTION 10

Cheat sheet & resources

Concept	One-liner
Anthropic	Company behind Claude; safety-first; founded by ex-OpenAI people
LLM	Trained brain that predicts the next token using context
Token	≈ ¾ of a word; the unit of cost and limits
Haiku / Sonnet / Opus	Fast & cheap / balanced default / most powerful
Extended thinking	Model thinks longer before answering; more tokens
Claude Desktop	Chat app for Windows; talks, can't act on your PC
Terminal (PowerShell)	Text control panel of your PC
Claude Code	Claude + terminal access = brain with hands
Agentic loop	Think → Act → Observe → Check → repeat

Resources

- Course repo: `github.com/mayank953/Youtube` → *Claude Code Crash Course* (mindmap PDF, demo files in `TestClaudeCode/`, final `Task-Manager-App/` project with agents, hooks & skills).
- Claude Code docs: `code.claude.com/docs`
- Models overview: `docs.claude.com` → Models overview (check current versions here).
- Claude Desktop for Windows: `claude.ai/download`

→ **Next: Video 2 — Installation**
Installing Claude Code on Windows, fixing the 'claude is not recognised' PATH issue, `claude doctor`, choosing your model, and Terminal vs VS Code extension. Send the transcript when ready.

Self-check: can you explain to a friend, without notes, why Claude Code needs to be installed and what the agentic loop is? If yes — Video 1 is done.

CLAUDE CODE CRASH COURSE · VIDEO 2 / 5

Installation & Setup
Windows Edition

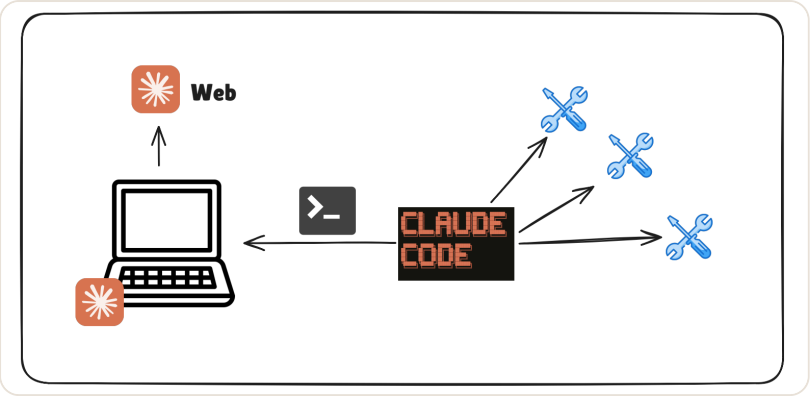
Claude Code ko Windows par install, authenticate aur run karna — PATH errors, tokens/cost, VS Code workflow aur pehle 3 hands-on prompts. AI engineer ke angle se samjhaya gaya.

01 Big Picture: Brain + Hands
02 Claude Code chalane ke 3 tareeqe
03 Windows par Install
04 "claude not recognized" — PATH Fix
05 Pehla Run: Login, Trust, Theme
06 Payment: Subscription vs API
07 Terminal UI ko samjhna
08 Tokens, Context & Cost
09 VS Code Extension
10 Hands-on: 3 Prompts, 3 Files
11 AI Engineer Corner
12 FAQ, Cheat Sheet & Links

Source: Mayank ka YouTube video (Day 2) + course repo *mayank953/Youtube › Claude Code Crash Course* + official docs (code.claude.com). Diagrams course repo se liye gaye hain. Notes date: 21 Oct 2026.

1 Big Picture: Claude = Brain, Claude Code = Brain + Hands

Claude (chat / web) sirf ek **brain** hai — code likh kar de sakta hai, lekin tumhari machine par file bana nahi sakta, run nahi kar sakta. **Claude Code** wohi brain hai jisko **haath (tools)** mil gaye hain: woh tumhare **terminal** ke through tumhare computer par kaam karta hai.



Course diagram — Laptop ↔ Terminal ↔ Claude Code ↔ Tools (🔧 = haath)

"Haath" = Tools

📖 Read

Folder ki files dekhna / parhna (code, docs — aur haan, bank statement bhi agar folder me pari ho!)

✍️ Write / Edit

Nayi file banana, existing file edit karna, delete karna

⚡ Execute

Shell commands chalana — Windows par **PowerShell** ya **Git Bash** (python run, tests, pip install...)

Agent Loop — har task ke peeche yahi hota hai

1. Think
Kya karna hai? Kaunsa tool?

→

2. Tool call
e.g. `Write hello.txt`

→

3. Execute
Tumhari approval ke baad machine par

→

4. Verify
Result check, error ho to khud fix

↻

🧠 AI Engineer lens

Yeh exactly wohi **tool-use / function-calling agent** pattern hai jo tum apne LLM apps me banate ho: model ko tool definitions milti hain → model `tool_use` return karta hai → harness (Claude Code) us tool ko locally execute karta hai → result wapas model ko jata hai → loop chalta hai jab tak task complete na ho. Claude Code = ek production-grade **agent harness**.

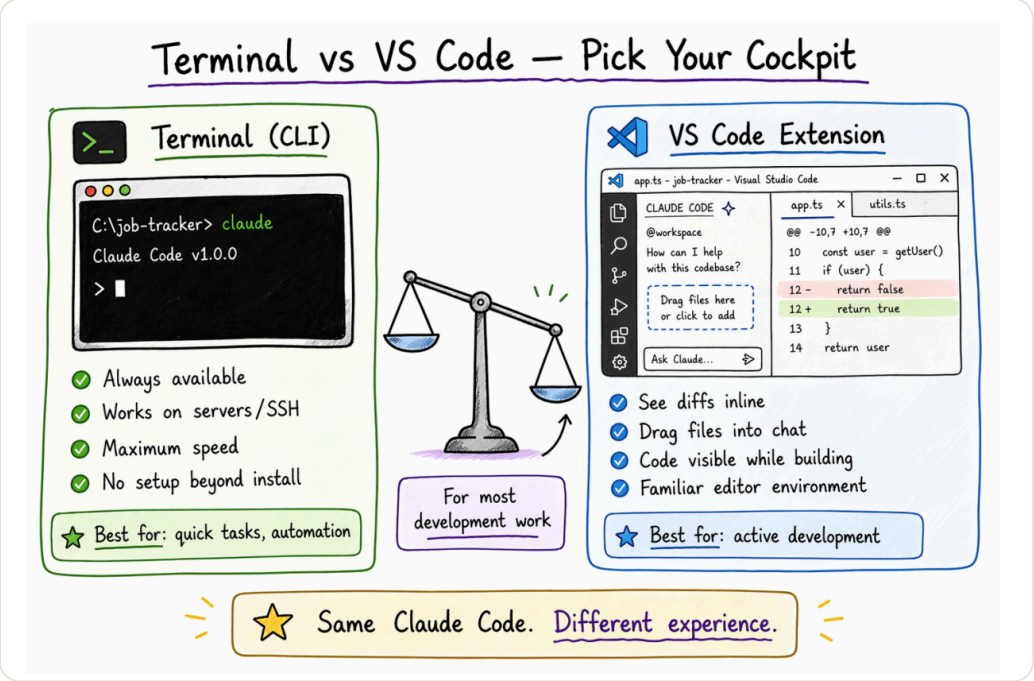
⚠️ Isi liye "Trust this folder?" poochta hai

Jis folder me Claude Code chalaoge, woh wahan read, write aur execute kar sakta hai. Isliye hamesha **project ke apne folder** me chalao — kabhi `C:\Users\You` (home) ya poore drive par nahi.

2 Claude Code chalane ke 3 tareeqe

Engine (brain + tools) ek hi hai — sirf **interface** badalta hai.

Tareeqa	Kya hai	Best for	Video ki rai
1. Claude Desktop app GUI	Claude app me left side par **Code** tab (Pro/Max par dikhta hai). Folder select karo, buttons se kaam. Koi terminal nahi.	Non-tech log, quick try	Kam recommend (video me error bhi aaya)
2. Terminal (CLI) Core	PowerShell/CMD/Windows Terminal me `claude` type karo. Sab features yahin pehle aate hain.	Speed, automation, servers (AWS/VPS par bhi chalta hai)	✅ Install isi se
3. IDE Extension Daily dev	VS Code / Cursor / Antigravity / JetBrains me official extension. Diffs side-by-side dikhte hain.	Real projects, active development	✅ Most preferred



"Same Claude Code. Different experience." — Terminal vs VS Code

> **💡 Key point**
> Extension bhi andar se wahi installed Claude Code use karta hai. Ek baar CLI install kar lo → har jagah (VS Code ka built-in terminal, Cursor, Windows Terminal) `claude` chal jayega.

3 Windows par Install

Requirements (official docs)

✅ **Zaroori**

- Windows 10 (1809+) / 11
- 4 GB+ RAM, x64 ya ARM64
- Internet connection
- Pro / Max / Team / Enterprise ya Console (API) account — **Free plan me Claude Code nahi**

⭐ **Recommended**

- **Git for Windows** — isse Claude ko **Git Bash** milta hai (Bash tool). Na ho to Claude PowerShell tool use karta hai.
- **Windows Terminal** (Microsoft Store) — tutorials wala Mac/Linux jaisa experience
- Administrator ki zaroorat **nahi**

Pehle pehchano: PowerShell ya CMD?

PowerShell prompt `PS C:\Users\You>` (shuru me **PS**) — Start → "PowerShell" ya Windows Terminal. CMD prompt `C:\Users\You>` — `Win+R` → `cmd`.

Install commands — sirf EK chalao

```
OPTION A · POWERSHELL (RECOMMENDED)
irm https://claude.ai/install.ps1 | iex

OPTION B · CMD
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del install.cmd

OPTION C · WINGET (AUTO-UPDATE NAHI HOTA)
winget install Anthropic.ClaudeCode   # update: winget upgrade Anthropic.ClaudeCode
```

Command chalte waqt progress nahi dikhti — wait karo. Native install (A/B) background me **khud update** hota rehta hai.

Verify karo (naya terminal window khol kar)

```
POWERSHELL
claude --version     # e.g. 2.1.xxx (Claude Code)
claude doctor        # health check: install, auto-update, settings errors
```

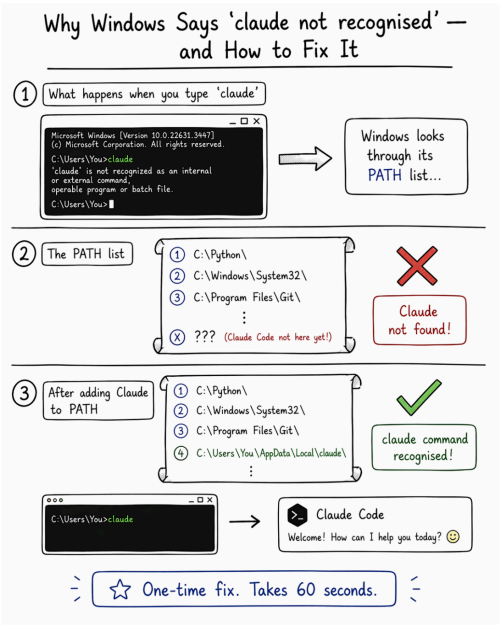
> ℹ️ **Node.js ka confusion (video vs docs)**
>
> Video me kaha gaya ke Node chahiye. Asal me **native installer (A/B/C) ko Node.js ki zaroorat nahi**. Node sirf tab chahiye jab tum npm wala tareeqa use karo: `npm install -g @anthropic-ai/claude-code` (Node 22+). AI engineer ho to Node waise bhi rakho — MCP servers waghaira me kaam aayega.

Common install errors

95% fix: internet → claude doctor → naya terminal

Error	Matlab / Fix
`The token '&&' is not a valid statement separator`	Tum **PowerShell** me CMD wala command chala rahe ho → Option A use karo
`'irm' is not recognized…`	Tum **CMD** me ho → Option B use karo ya PowerShell kholo
`'claude' is not recognized…`	Install ho gaya, lekin folder **PATH** me nahi → Section 4
Bash tool kaam nahi kar raha	Git for Windows install karo (ya PowerShell tool par chalne do)

4 "claude is not recognized" — PATH Fix



Course diagram. Note: native installer ka actual folder `%USERPROFILE%\.local\bin` hai (diagram me example path different hai).

Concept

Jab tum `claude` type karte ho, Windows **PATH** naam ki list ke har folder me `claude.exe` dhoondta hai. Agar install wala folder us list me nahi → "not recognized".

Installer `claude.exe` yahan rakhta hai:

```
%USERPROFILE%\.local\bin\claude.exe
# = C:\Users\<YourName>\.local\bin
```

Fix (GUI tareeqa — 60 seconds)

1. Start menu → **"Edit environment variables for your account"**
2. **User variables** me **Path** select → **Edit**
3. **New** → paste: `%USERPROFILE%\.local\bin`
4. OK → OK → OK
5. **Saare terminal band karo**, naya kholo → `claude --version`

Fix (PowerShell one-liner)

POWERSHELL · USER PATH ME ADD

```
$p = [Environment]::GetEnvironmentVariable("Path","User")
[Environment]::SetEnvironmentVariable("Path",
"$p;$env:USERPROFILE\.local\bin", "User")
```

Phir naya terminal kholo. (VS Code bhi restart karo warna uska integrated terminal purana PATH yaad rakhta hai.)

🧠 AI Engineer lens

Yeh wahi PATH concept hai jiski wajah se `python` / `pip` / `uv` "not recognized" aata hai. Rule: **naya tool install → naya terminal**. Environment variables process start hone par load hote hain.

5 Pehla Run: Theme → Login → Trust

Golden rule: pehle project folder me jao, *phir* `claude` chalao. (Video me pehle home folder me chala diya tha — phir exit karke folder me gaye.)

```powershell
mkdir $HOME\Desktop\test-claude-code
cd $HOME\Desktop\test-claude-code
claude
```

① Theme Text style choose (e.g. "Auto-match terminal") → **② Login method** Subscription / Console / 3rd-party → **③ Browser authorize** "Login successful" → Enter → **④ Terminal setting** Recommended → Yes → **⑤ Trust folder?** Apna folder → Yes

🔐 Prompt-injection warning

Login ke baad Claude batata hai ke woh prompt injection ki wajah se ghalti kar sakta hai. Matlab: kisi file/webpage ke andar chhupi "instructions" Claude ko gumraah kar sakti hain. Unknown repos par khas ehtiyat.

↩️ Trust par "No" daba diya?

Koi masla nahi. Claude Code band karo, usi folder se dobara `claude` chalao — dobara poochega. Yeh one-time decision nahi.

Kaam ke commands

Command	Kaam
`claude`	Current folder me interactive session
`/login` · `/logout`	Account / payment method badalna
`/model`	Model switch (Haiku ↔ Sonnet ↔ Opus)
`/context`	Context window kitna bhara hai, breakdown
`/clear`	Naya conversation (purana context hata do)
`exit` / `Ctrl+C` ×2	Session se bahar
`claude --verbose`	Andar ki details + token counts dikhata hai (learning ke liye best)
`claude doctor` · `claude update`	Health check · manual update (session ke bahar chalao)

Slash commands, modes aur models ki full detail Video 3 me aayegi.

6 Payment: Claude Code free nahi hai

LLM chalana Anthropic ke liye mehenga hai, isliye login ke waqt batana parta hai ke **pay kaise karoge**. 3 options:

① Subscription

Pro / Max / Team

claude.ai account se login. Fixed monthly (Pro ≈ $20). Usage limits: **~5-hour session window** + **weekly limit**. Settings → Usage me % dekh sakte ho; "extra usage" credits bhi add ho sakte hain.

🏋️ **Gym membership:** fixed fee, lekin roz limited time.

② Console / API key

Pay-as-you-go

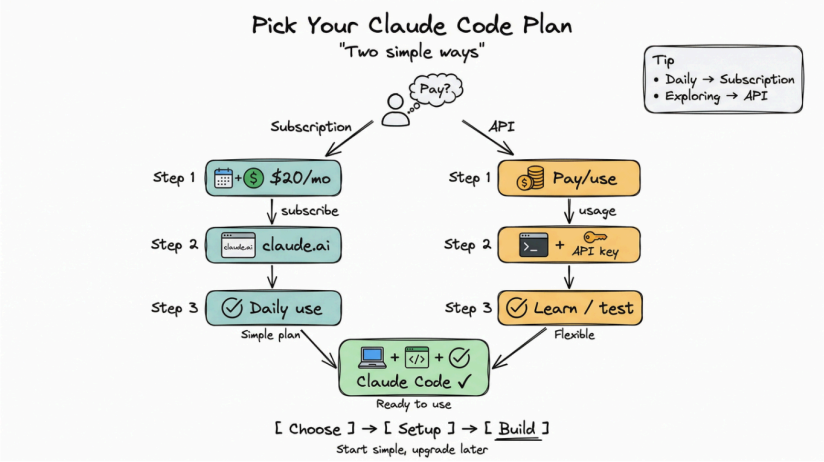
platform.claude.com par credits khareedo ($5 bhi kaafi start ke liye). Login par Claude khud key bana leta hai, ya `ANTHROPIC_API_KEY` env var set karo. Har token ka paisa.

🎟️ **Per-visit ticket:** jitna jao, utna do.

③ Third-party Cloud

Amazon Bedrock, Google Cloud (Vertex/Agent Platform), Microsoft Foundry — company ke cloud billing se. Local models (Ollama + Llama/Qwen) bhi jud sakte hain, lekin...

🏎️ **"Ferrari with a cycle engine"** — asal taqat brain me hai.

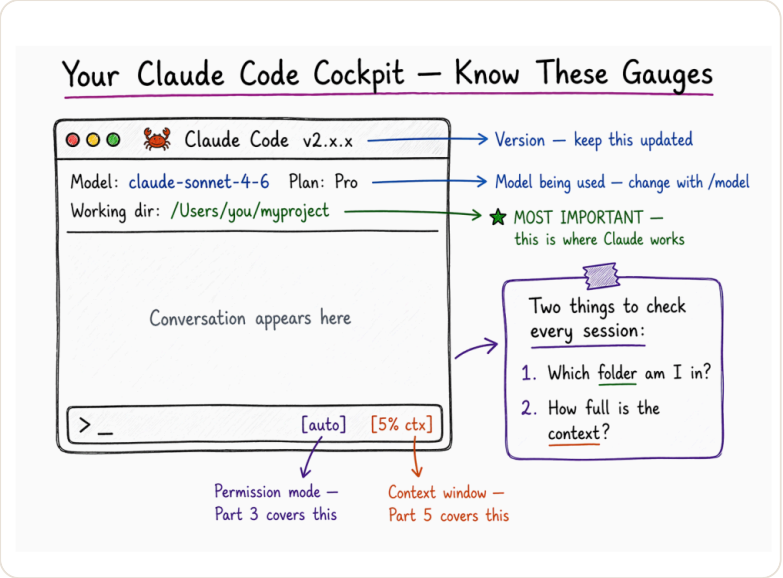


Course diagram — Daily use → Subscription · Learning / exploring → API

🧠 AI Engineer recommendation

- **Daily coding** ke liye subscription sasta parta hai (heavy agentic usage API par jaldi mehenga ho jata hai).
- **Experiments / apne apps** ke liye Console key alag rakho — wahan har request ka cost dikhta hai, cost-optimization seekhne ka best tareeqa.
- Companies ab devs ko monthly $300–500 Claude budget deti hain — aur expect karti hain ke tum **optimize** karo, $2,000 ke credits na jala do.
- API key kabhi chat/prompt me paste ya git me commit mat karo — `.env` + `.gitignore`.

7 Terminal UI ko samjhna — Cockpit gauges



Course diagram — har session me 2 cheezein check karo: **kis folder me hoon** aur **context kitna bhara hai**

Gauge	Kya batata hai
Version	Claude Code version — native install khud update hota hai
Model + Plan	Kaunsa model chal raha hai, aur billing kis se (Pro / API) — `/model` se badlo
Working directory ⭐	Sabse important — Claude sirf isi folder me kaam karega
Permission mode	Edits ke liye har baar poochega ya auto-accept (Video 3)
Context %	Context window kitni bhari hai (Video 5 me detail)

Permissions — har action par tum boss ho

✍️ File write / edit

- **Yes** — sirf is baar
- **Yes, allow all edits this session** — is session me dobara nahi poochega
- **No** — aur Claude ko batao kya alag karna hai

⚡ Bash / PowerShell command

- Har command dikhata hai (jaise `ls`, `python greet.py`)
- "Always allow" sirf safe commands ke liye
- Learning phase me **har command parho** — samajh aayega Claude kya kar raha hai, aur kuch delete to nahi ho raha

🔌 LSP plugin prompt

Pehli Python file par Claude ek LSP (Language Server Protocol) plugin suggest kar sakta hai — isse "go to definition" aur error checking jaisi code intelligence milti hai. Install kar lena theek hai.

8 Tokens, Context & Cost — sabse important concept

1 · Token kya hai?

Text ke tukre. Takreeban **1 token ≈ ¾ word**. `"Hello, world!"` → Hello · , · world · !

2 · Bill kaise banta hai?

Cost = Input tokens + Output tokens (dono ka rate alag, output mehenga)

3 · Zyada tokens

→ zyada cost, aur context jaldi bharta hai

Surprise: ek "hello" file = ~17,000 tokens!

Video me `claude --verbose` se ek 10-line Python file banwai → **16,991 tokens**. `/context` ne breakdown dikhaya:

Hissa	Tokens (approx)	Kya hai
System prompt	~6k	"Tum Claude Code ho, achha code likho..." wali instructions
System tools	~8k	Read, Write, Edit, Bash waghaira ki definitions
Skills / memory / messages	baqi	Tumhara prompt + conversation history

> 💡 **Kyun itne tokens?**
>
> LLM **stateless** hai — use kuch yaad nahi rehta. Har request par poora system prompt + tool definitions + chat history **dobara bheja jata hai**. Isliye lambi chat = har naya message mehenga. Topic badlo to `/clear` karo.

Current model pricing (Claude API docs, Oct 2026)

Model	Input / 1M tokens	Output / 1M tokens	Kab use karo
Haiku 5.5	from $0.10	from $0.50	Simple, repetitive, sasta kaam
Sonnet 5.5	$2	$10	Default workhorse — zyada tar coding
Opus 5.5	$4	$20	Planning, architecture, mushkil bugs

Video purane models (Sonnet 4.6, $3/$15, 200k context) dikhata hai. Prices badalti rehti hain — hamesha pricing page check karo.

```
EXAMPLE CALCULATION · SONNET 5.5
1M input tokens  × $2  = $2
1M output tokens × $10 = $10
                 Total = $12   # output 5x mehenga — verbose answers = zyada bill
```

> 🧠 **AI Engineer lens — model routing**
>
> Apne LLM apps me bhi yahi pattern: **Opus se plan**, **Sonnet se build**, **Haiku se simple/bulk kaam**. Claude Code me `/model` se switch karo. Bar-bar repeat hone wala system prompt API me **prompt caching** se kaafi sasta hota hai — Claude Code yeh khud karta hai; apne apps me tum bhi karo.

9 VS Code Extension — real projects ka cockpit

Install

1. VS Code → Extensions (`Ctrl+Shift+X`) → search **"Claude Code"**
2. **Sirf official** install karo: publisher **Anthropic**, millions installs (video me ~11M). Fake/look-alike extensions tumhari API key ya data chura sakti hain.
3. Top-right / sidebar me Claude icon → chat panel khul jata hai (GitHub Copilot jaisa)

Cursor, Antigravity, JetBrains — sab me process same hai.

Terminal ke muqable kya extra milta hai?

- **Inline diffs** — file likhne se pehle dikhata hai kya badlega (red/green)
- Files **drag-drop** karke chat me do, ya `@file` mention
- Jo file open hai woh context me hoti hai (video me `links.txt`)

- **Multiple Claude instances / tabs** ek saath
- Upload, "Add context", "Browse the web" options
- VS Code ke integrated terminal me `claude` CLI bhi chalta hai

✅ Workflow

VS Code me **File → Open Folder** se project kholo → Claude panel → model choose (practice ke liye Haiku, usage bachane ke liye) → prompt. Folder hi Claude ki working directory banta hai.

⚠️ Windows note

PATH change ke baad VS Code ko **poora restart** karo, warna integrated terminal me `claude` "not recognized" aa sakta hai.

10 Hands-on: 3 Prompts, 3 Files, Zero Manual Coding

Folder: `my-first-session`. Har prompt pichhle se mushkil — dekho agent loop kaise grow karta hai.

Prompt 1 — Sirf Write 1 tool

```
PROMPT
Create a file hello.txt containing the message "My name is <your name>"
```

Claude ne **Write tool** chuna → diff dikhaya → approval → file bani → path repeat karke **verify** kiya. Poora agent loop < 5 seconds.

Prompt 2 — Read + Write + Execute 3 tools

```
PROMPT
Look at what's currently in this folder, then create a Python script called greet.py
that asks the user for their name and prints "Hello <name>! Welcome to Claude Code."
After creating it, run it and show me the output.
```

Pehle folder list kiya (`ls` / `dir` jaisa command) → file likhi → run ki. Generated code (course repo se):

```
GREET.PY
name = input("What is your name? ")
print(f"Hello {name}! Welcome to Claude Code.")
```

Prompt 3 — Multi-step + self-testing agentic

💡 Claude ne **khud** error case bhi test kiya — "plan → build → test → fix". Achha prompt = clear task + **acceptance criteria**.

```
PROMPT
Create a small Python script called number_facts.py that takes a number as a
command-line argument. Check if it's prime, a perfect square, and a Fibonacci number.
Print all three results neatly formatted. Then run it with 144 and show me the output.
Then make sure it handles the case when no argument is given (helpful error message).
```

```
NUMBER_FACTS.PY · CORE LOGIC (COURSE REPO)
def is_perfect_square(n):
    if n < 0: return False
    s = int(math.sqrt(n)); return s * s == n

def is_fibonacci(n):          # math trick: 5n²±4 perfect square ho to n Fibonacci hai
    return n >= 0 and (is_perfect_square(5*n*n + 4) or is_perfect_square(5*n*n - 4))

# + is_prime(n): sirf √n tak odd divisors check · + sys.argv validation → error message
```

```
WINDOWS PAR KHUD RUN KARO (POWERSHELL)
python number_facts.py 144     # Windows par "python3" nahi, "python" ya "py"
=== Number Facts for 144 ===
Prime:          False
Perfect Square: True
Fibonacci:      True
python number_facts.py
Error: Please provide a number as an argument.
```

11 AI Engineer Corner — is video ka asal faida

Apne kaam me kahan use karo

🔧 Daily AI-engineering tasks

- FastAPI / Flask endpoints for LLM apps
- RAG pipeline scaffolding: loaders, chunking, embeddings, vector DB
- Eval scripts — golden datasets, scoring, reports
- Data cleaning / CSV→JSONL fine-tune prep
- Unit tests likhwana aur chalwana
- Webhook bots (WhatsApp/Telegram) ka boilerplate + debugging

🛡️ Safe working habits

- Har project ka **alag folder**, wahin se `claude`
- Bade change se pehle `git commit` — undo aasaan
- Keys `.env` me; Claude ko key ki value kabhi mat do
- Delete/deploy jaise irreversible commands khud parho
- Naya task = `/clear` (sasta + focused)
- Pehle chhota prompt, phir scope barhao

Concepts jo tumhare apne LLM apps par bhi lagte hain

Claude Code me dekha	Tumhare LLM apps me matlab
Tools = haath	Function calling / tool use — model decide karta hai, tumhara code execute karta hai
Permission prompts	Human-in-the-loop guardrails dangerous actions par
~14k tokens overhead	System prompt + tool schemas ka hidden cost — tools kam rakho, descriptions tight
Stateless + history resend	Conversation memory = tokens; summarize / trim karo
Opus / Sonnet / Haiku	Model routing by task difficulty → cost control
Prompt injection warning	Untrusted content (emails, webpages, user msgs) ko instructions mat banne do

> **🚀 Aage ke videos ka preview**
>
> Video 3: slash commands, modes, models · Aage: `CLAUDE.md` (project memory), Skills, Hooks, MCP (Claude ko Gmail/DB/APIs se jodna), Sub-agents, aur headless/scheduled automation.

Practice checklist

- ☐ Git for Windows + Windows Terminal install
- ☐ PowerShell se Claude Code install → `claude --version` → `claude doctor`
- ☐ Test folder banao, login (subscription ya $5 API credits), trust folder
- ☐ `claude --verbose` me ek file banwao, phir `/context` dekho
- ☐ VS Code extension (official) install, wahi 3 prompts chalao
- ☐ Settings → Usage me dekho kitna % gaya

12 FAQ, Cheat Sheet & Links

FAQ (course repo ke Questions.md se)

Sawal	Jawab
Install command kaam nahi kar raha?	Internet → `claude doctor` → naya terminal. Zyada tar issues yahin hal.
Windows par WSL use karun?	Zaroori nahi — native PowerShell install bilkul theek. WSL tab jab Linux toolchain ya sandboxing chahiye.
Trust par "No" daba diya?	Band karo, usi folder se dobara chalao.
Kaunsa model se start karun?	Sonnet (default workhorse). Bachat: Haiku. Max capability: Opus.
Git zaroori hai?	Claude Code ke liye nahi, lekin Windows par Git for Windows (Git Bash) recommended; aur commits/branches me Claude madad karta hai.

One-page cheat sheet

```
WINDOWS · POWERSHELL
# Install / verify
irm https://claude.ai/install.ps1 | iex
claude --version
claude doctor
claude update

# PATH fix (agar "not recognized")
$p = [Environment]::GetEnvironmentVariable("Path","User")
[Environment]::SetEnvironmentVariable("Path", "$p;$env:USERPROFILE\.local\bin", "User")

# Run in a project
cd $HOME\Desktop\my-project
claude                # interactive
claude --verbose      # tokens/details dekhne ke liye

# Andar ke commands
/model   /context   /clear   /login   exit
```

Links (course repo + official)

- Course repo: github.com/mayank953/Youtube › Claude Code Crash Course
- Claude Code docs (overview / VS Code): code.claude.com/docs/en/overview
- Setup & Windows details: code.claude.com/docs/en/setup
- Models & pricing: platform.claude.com/docs/en/about-claude/models/overview
- API keys (Console): platform.claude.com/settings/keys
- Claude Code GitHub: github.com/anthropics/claude-code
- Windows Terminal (Microsoft Store): apps.microsoft.com/detail/9n0dx20hk701
- Git for Windows: git-scm.com/downloads/win

CLAUDE CODE CRASH COURSE · VIDEO 3 OF 5

Slash Commands, Sessions & Permission Modes

Claude Code ko "chat box" ki tarah nahi, ek pro engineer ki tarah chalana — model, context, sessions aur safety ka poora control.

`/model` `/effort` `/usage` `/context` `/compact` `/clear`
`/rewind` `/branch` `/rename` `/resume` `/exit` `Shift+Tab`
`plan mode` `auto mode`

AI Engineer Notes · Windows edition

Verified against Claude Code docs · 21 Oct 2026

IS PDF MEIN

Contents

01 Big picture — "Ferrari ki chaabi"

02 Slash commands kya hain

03 Model control — /model, /effort, /fast

04 Tokens, cost & usage

05 Context window — har message pe kya jata hai

06 /compact vs /clear

07 /rewind & checkpoints

08 Sessions — rename, exit, resume, branch

09 Permission modes

10 Plan mode workflow

11 Baaki useful commands

12 AI Engineer playbook

13 Video said → Now (corrections)

14 Revision cheat sheet

A Appendix — 74 commands, 9 categories

IS PDF KO KAISE PARHNA HAI

Har chapter mein 4 tarah ke boxes hain: **Seedhi baat** (Roman Urdu mein simple summary), **Engineer tip** (real projects ke liye), **Careful** (galti jo nuksaan kar sakti hai), aur **Video said → Now** (jahan video ke baad Claude Code badal chuka hai — docs se verify kiya gaya). Sab commands Windows (PowerShell / VS Code) ke hisaab se likhe hain.

COURSE RESOURCES (GITHUB REPO)

Video 3 se related repo mein ye cheezein hain: `TestClaudeCode/claude-slash-commands-guide.html` (instructor ki 70+ commands wali website — Appendix A isi se bani hai), demo files jo video mein bani (`kanban.html` , `add.py` , `hello.py` , `helloMayank.txt`), aur course board PDF jiske "Sessions, Commands, Modes" section ke diagrams is PDF mein lagaye hain.

OVERVIEW

01 The big picture

Video 2 mein Claude Code install ho gaya. Lekin instructor ki analogy: **aap ke haath mein Ferrari ki chaabi hai, magar controls ka pata nahi.** Zyada tar log Claude Code ko sirf "kuch likho, jawab lo" ki tarah use karte hain. Video 3 ka goal hai ke aap cockpit ke har control ko samjhein:

1 · Slash commands

Claude Code ke "keyboard shortcuts" — model, context, cost, sessions control karna, bina conversation tokens kharch kiye.

2 · Context management

Har message pe Claude ko kya bheja jata hai, aur context ko `/compact`, `/clear`, `/rewind` se kaise saaf rakhna hai.

3 · Sessions

Har conversation ek saved session hai — naam do, theek se exit karo, kal resume karo, branch karke experiment karo.

4 · Permission modes

Claude kitna khud kar sakta hai bina poochhe — Manual, Accept edits, Plan, Auto, Bypass.

Instructor ne terminal aur VS Code extension dono dikhaye. Real projects ke liye **terminal ya VS Code** main jagah hai; Claude Desktop sab se kam use hoga. Commands dono jagah same kaam karte hain (kuch commands sirf terminal mein milte hain).

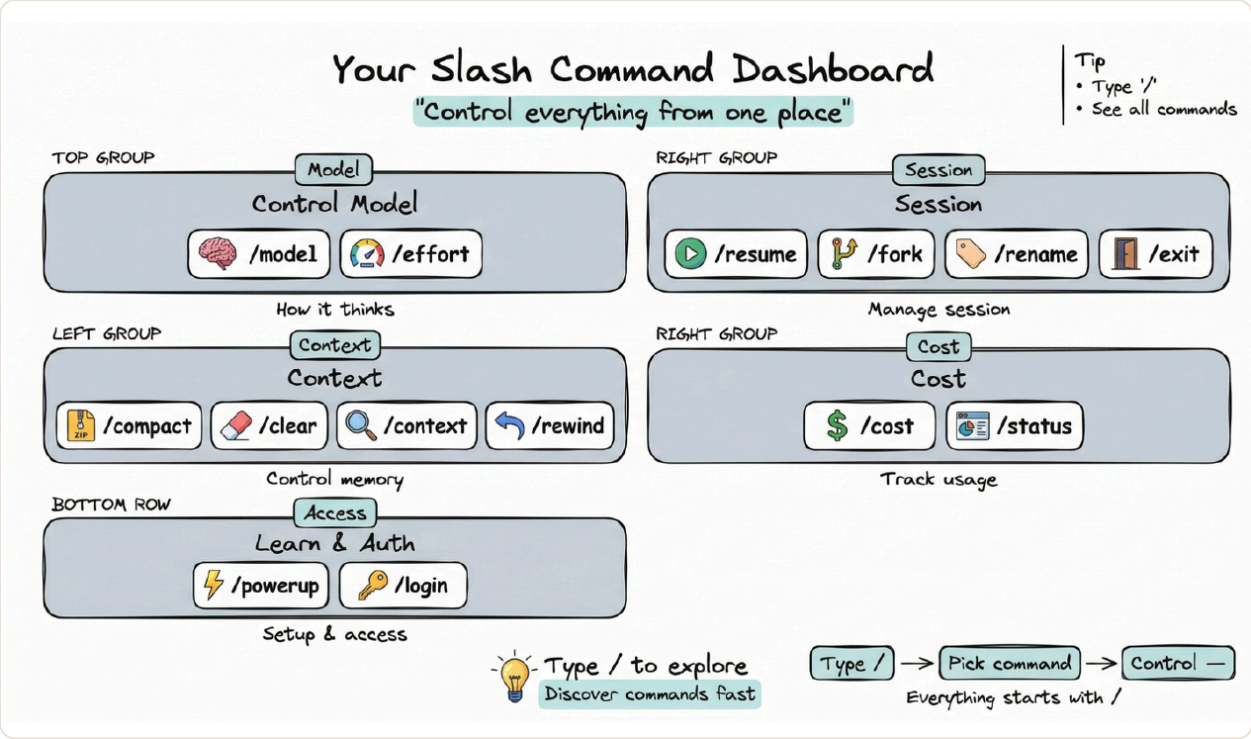
> **SEEDHI BAAT**
>
> Jo banda sirf prompt likhta hai aur jo banda `/context` dekh ke, sahi model choose karke, session ka naam rakh ke, plan mode mein kaam shuru karta hai — dono ek hi tool use kar rahe hain, lekin result aur cost mein zameen aasmaan ka farq hai. Ye video us farq ke baare mein hai.

CONCEPT

02 Slash commands kya hain

Prompt box mein `/` type karo — commands ki list khul jati hai. Aage letters likho to filter ho jati hai. Jaise **Ctrl+C / Ctrl+V** copy-paste ke shortcut hain, waise hi slash commands Claude Code ki capabilities pe **direct control** dete hain.

- Command sirf **message ke shuru** mein recognize hota hai. Command ke baad likha text uske arguments ban jata hai (jaise `/compact sirf bug wali baat yaad rakho`).
- Zyada tar commands Claude Code khud chalata hai — ye model ko prompt ban kar nahi jaate, isliye conversation ka context nahi khaate.
- Agar Claude jawab de raha ho aur aap command bhejo, to wo queue ho jata hai; kuch commands (`/status`, `/usage`) foran chal jaate hain.



Course board: "Your Slash Command Dashboard" — video mein cover hone wale commands 5 groups mein

Group	Commands	Kaam
Model	`/model` `/effort` `/fast`	Claude kaise sochta hai — kaunsa brain, kitni der soche
Context	`/context` `/compact` `/clear` `/rewind`	Memory (context window) control karna
Session	`/rename` `/resume` `/branch` `/exit`	Conversations ko manage karna
Cost	`/usage` (`/cost`, `/stats`) `/status`	Kharcha aur limits track karna
Access / Learn	`/login` `/logout` `/powerup`	Account aur seekhna

> **ENGINEER TIP**
>
> Poori list Appendix A mein hai (74 commands, 9 categories). Sab yaad karne ki zaroorat nahi — upar wale ~15 commands roz use honge. Baaki ke liye `/help` ya `/` type karke filter karo.

COMMANDS

03 Model control — /model, /effort, /fast

/model — dimagh badlo

`/model` chalao to picker khulta hai: Opus, Sonnet, Haiku (aur jo aap ke plan pe available hon). Directly bhi de sakte ho: `/model sonnet`. Ye aap ka default bhi save kar deta hai; sirf current session ke liye badalna ho to picker mein row pe `s` dabao.

Instructor ki "engineering team" strategy



Rule: sahi model ka matlab sab se taqatwar model nahi — **kaam ke hisaab se model**. Har cheez Opus pe chalana limits jaldi khatam karta hai (instructor: Pro plan pe Opus ke 4–5 heavy messages ke baad "baad mein aana" aa sakta hai).

CAREFUL — BAAR BAAR MODEL MAT BADLO

Claude Code aap ki conversation ko **prompt cache** mein rakhta hai (pichle messages dobara poore paise pe process nahi hote). Beech mein model badla to naya model poori history shuru se parhta hai — isliye switch karte waqt warning aati hai. Model ko **phase ke shuru** mein badlo (planning → building), har doosre message pe nahi.

VIDEO SAID → NOW

Video mein "Sonnet 4.6", Opus 4.6 aur Haiku 200K context dikhaye gaye. Ab (Oct 2026, Anthropic account pe): `default` = **Opus 5.5**, `sonnet` = Sonnet 5.5, `haiku` = Haiku 5.5. Current models (Opus 4.7+, Sonnet 5+, Haiku 5.5, Fable) pe **1M token context by default** hai. Ek kaam ka alias bhi hai: `opusplan` = plan mode mein Opus, execution mein Sonnet — instructor ki strategy automatic.

/effort — kitni der soche

Effort ek "difficulty slider" hai: low = jaldi jawab, max = deep dive. Zyada effort = zyada thinking aur lamba agentic loop = zyada tokens.

Level	Kab use karein
`low`	Simple sawal, formatting, choti edits
`medium`	Roz ka kaam — Opus/Sonnet/Haiku 5.5 pe yahi default hai
`high` / `xhigh`	Multi-file features, tricky bugs, architecture
`max`	Sab se mushkil problems — sirf is session ke liye lagta hai
`auto`	Claude khud decide kare

/fast — same model, tez output

`/fast` fast mode on/off karta hai: model wahi rehta hai, output tez aata hai, lekin ye zyada usage khaata hai. Demo ke time ya jaldi iteration mein kaam ka hai.

SEEDHI BAAT

Plan Opus se, build Sonnet se, chhota kaam Haiku se. Effort normal kaam pe medium, mushkil pe high. Model phase badalne pe badlo, beech mein nahi — warna cache toot jata hai aur paise lagte hain.

COMMANDS

04 Tokens, cost & usage

Token = currency. Video 2 ka concept yaad karo (1 token ≈ ¾ word). Jo aap bhejte ho wo **input tokens**, jo Claude likhta/sochta hai wo **output tokens**. Output tokens mehnge hote hain kyun ke model input ke upar soch kar naya content generate karta hai.

/usage (aliases: /cost, /stats)

Video mein `/cost` chalaya to total cost, API duration, wall time aur code changes (lines added/removed) dikhe. Ek chhota task ("2+2 ka Python script") karne ke baad dobara chalaya to number barh gaya — **har run ke baad cost barhti hai**.

- **Session tab:** is session ka cost, tokens, cache hits
- **Plan limits:** subscription pe current session % aur current week % (video: session 2%, week 18%)
- **Stats tab:** favorite model, total tokens, activity graph

VIDEO SAID → NOW

Video mein `/cost` aur usage alag lag rahe the aur instructor ko lagta tha ke "combine ho gaye". Ab confirm: `/usage` asal command hai, `/cost` aur `/stats` uske **aliases** hain. `/status` alag hai — version, model, account aur connectivity dikhata hai.

ENGINEER TIP — SUBSCRIPTION VS API

Pro/Max subscription pe bhi `/usage` dollar cost dikhata hai — ye "agar API pe hota to itna lagta" wala andaza hai, bill nahi. Asli cheez aap ke liye **plan limit %** hai. Company mein API key pe kaam karoge to yahi number asli bill ban jata hai — isliye heavy run (refactor, test generation, agent loop) ke baad hamesha `/usage` check karne ki aadat dalo.

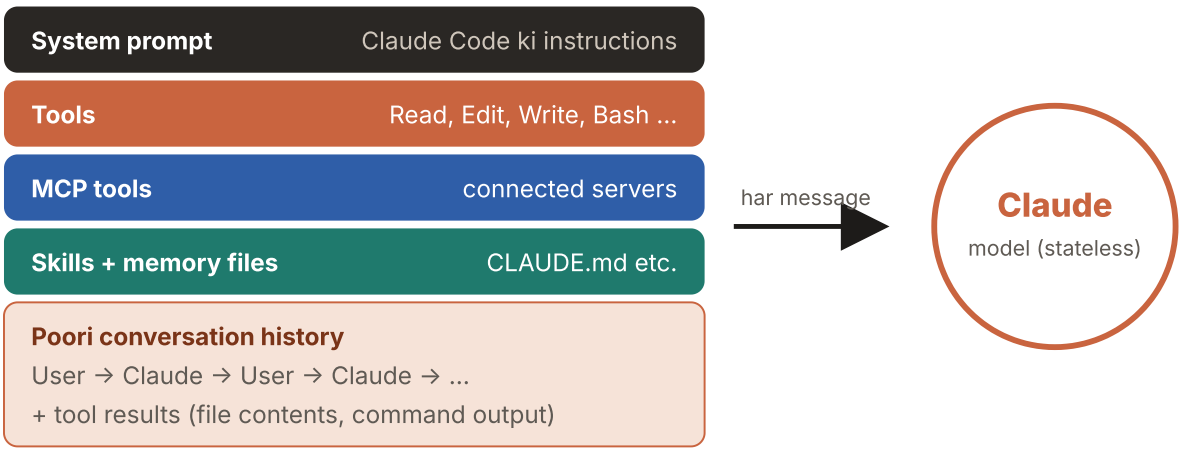
SEEDHI BAAT

`/usage` = receipt. Bara kaam karwaya? Receipt dekho. Week ka % tez barh raha hai? Model chhota karo, effort kam karo, ya context saaf karo (agle chapters).

CORE CONCEPT

05 Context window — har message pe kya jata hai

Sab se important concept: **Claude ko "yaad" nahi rehta.** Ye nahi ke pichle 3 message use yaad hain aur sirf naya message jata hai. **Har baar** jab aap message bhejte ho, poora package dobara bheja jata hai:



Tools aur MCP kyun bhejte hain? Instructor ki misaal: aap ko furniture banana hai — pehle pata hona chahiye ke toolbox mein kya hai. Claude ko bhi bataya jata hai "ye tools aur connections tumhare paas hain", tabhi wo informed decision le sakta hai.

/context — dekho context kis ne khaya

Video mein sirf **"hi"** bhejne ke baad `/context` ne ~29K tokens dikhaye. Kyun? Kyun ke system prompt (~6K), system tools (~20K), MCP tools aur skills pehle se baithe hain. "hi" ka hissa to chand tokens hai.

Category	Kya hai	Aap kam kar sakte ho?
System prompt	Claude Code ki apni instructions	Nahi
System tools	Built-in tools ki definitions	Nahi (terminal vs VS Code mein thoda farq)
MCP tools	Connected MCP servers	**Haan** — fazool servers band karo
Memory files / skills	CLAUDE.md, skills ki descriptions	**Haan** — CLAUDE.md chhota rakho (Video 4)
Messages	Aap ki conversation + tool results	**Haan** — /compact, /clear, /rewind
Autocompact buffer	Compaction ke liye reserved jagah	—
Free space	Jo bacha hai	—

Kanban board banwane ke baad context ~45K se ~51K ho gaya — har kaam ke saath barhta hai.

> **CONTEXT = RAM**
> Jaise browser mein bohat tabs khol do to computer slow ho jata hai, waise hi context bharne pe Claude **slow, mehnga aur kam accurate** hota jata hai (bohat saari purani, ghair-zaroori info ke beech asal baat kho jati hai — isko "context rot" kehte hain). Pro engineers isi liye context ko saaf rakhte hain.

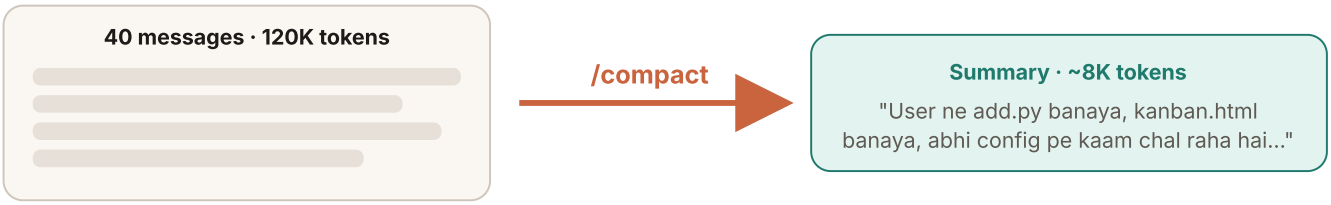
> **SEEDHI BAAT**
> Claude har message pe sab kuch dobara parhta hai. Jitni zyada history, utna zyada kharcha aur utni zyada confusion. `/context` se dekho, phir agla chapter wale tools se saaf karo.

COMMANDS

06 /compact vs /clear

/compact — "minutes of meeting"

Hum Claude se lambi, spelling-mistakes wali, 100 words ki baat 2000 words mein karte hain. `/compact` purani conversation ko ek **summary** se replace karta hai — jaise meeting ke baad poori recording ki jagah "minutes" rakh lete hain. Kaam ki baatein bachti hain, context free hota hai.



- `/compact` — poori conversation summarize karo
- `/compact sirf meri requests yaad rakho` — **focus instructions** ke saath (video: "just remember my requests and nothing else"; docs misaal: "focus only on the auth bug")
- **Auto-compact:** settings mein by default on — context bharne ke qareeb khud compact ho jata hai ("Compacting our conversation..." message yahi hai)

Video mein sirf 4 messages the, is liye bachat sirf ~600 tokens hui. Compact ka asal faida 15–40 messages wali lambi conversation pe hota hai. System prompt, tools, MCP pe compact ka koi asar nahi — sirf **messages** chhote hote hain.

> **VIDEO SAID → NOW**
>
> Video mein auto-compact "~170–180K pe" bataya gaya (200K window ke hisaab se). Ab 1M-context models pe auto-compact default ~967K tokens pe trigger hota hai. Isliye ab **manual** `/compact` aur bhi zaroori hai — 1M tak intezaar karoge to us se bohat pehle hi quality aur cost kharab ho chuke honge.

/clear — naya notebook

`/clear` (aliases `/reset`, `/new`) empty context ke saath **nayi conversation** shuru karta hai. Purani delete nahi hoti — disk pe save rehti hai aur `/resume` se wapas aa sakti hai. Video mein clear ke baad context sirf base ~43K (system + tools) reh gaya.

- `/clear release-prep` — jo session chhod rahe ho usko naam de do, taake picker mein mil jaye.

Situation	Use
Same task chal raha hai, lekin conversation lambi ho gayi	`/compact` (focus instructions ke saath)
Bilkul naya, unrelated task shuru kar rahe ho	`/clear`
Ek ghalat direction ka hissa hatana hai	`/rewind` (agla chapter)
Sirf ek chhota side-sawal poochhna hai	`/btw <sawal>` — main conversation mein add nahi hota

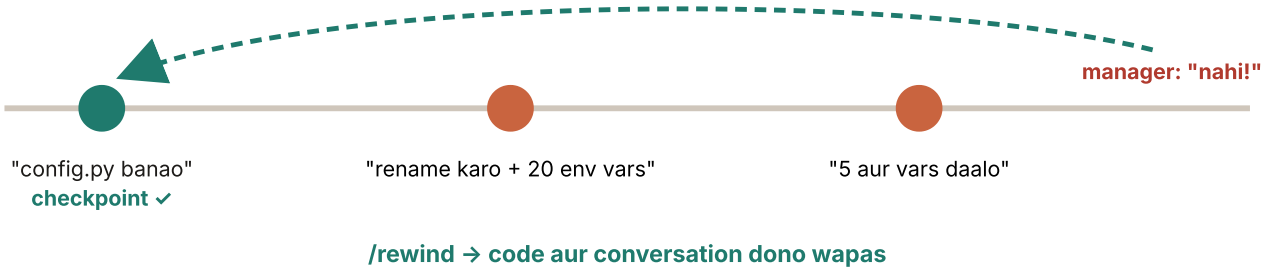
SEEDHI BAAT

Same kaam, lambi baat → **compact**. Naya kaam → **clear**. Ek kaam khatam, doosra shuru — beech mein `/clear` karna sab se sasti aur sab se acchi aadat hai.

COMMANDS

07 /rewind & checkpoints

Video game ka save point. Har prompt bhejne se pehle Claude Code aap ki files ki state ka **checkpoint** le leta hai. `/rewind` (aliases `/checkpoint`, `/undo`) — ya **khaali prompt pe Esc do baar** — menu kholta hai jahan se aap kisi pichle message pe wapas ja sakte ho. Ye poore coding session ka **Ctrl+Z** hai.



Rewind menu ke options

Option	Kya hota hai
Restore code and conversation	Files aur chat dono us point pe wapas
Restore conversation	Chat wapas, current code wese hi rahe
Restore code	Files wapas, chat wese hi rahe
Summarize from here	Us point se aage ka hissa summary bana do (targeted compact)
Summarize up to here	Us point se pehle ka hissa summary, baad wale messages intact

CAREFUL — VIDEO WALA "DATABASE_CONFIG KYUN REH GAYA?" KA JAWAB

Video mein rewind ke baad `config.py` to wapas aa gaya lekin `database_config.py` reh gaya, aur instructor ko wajah nahi pata thi. Sab se mumkin wajah docs mein hai: **checkpoints sirf Claude ke file-editing tools (Write/Edit) ki changes track karte hain.** File rename aam tor pe Bash command (`mv`) se hota hai, aur **Bash se ki gayi changes (rm, mv, cp) rewind nahi hotin.** Isi tarah aap ki manual edits aur doosre sessions ki edits bhi track nahi hotin.

- Checkpoints session ke saath save hote hain — resume karne ke baad bhi rewind chalta hai.
- Ek session mein last 100 checkpoints ke snapshots rakhe jaate hain; ~30 din baad cleanup.
- Agar `/clear` kiya tha, rewind menu ke top pe "previous session" entry hoti hai — clear se pehle wali conversation wapas.

ENGINEER TIP

Checkpoint **Git ka replacement nahi**. Rewind = session ke andar jaldi wala undo. Asli history = Git commits. Kaam ki aadat: kaam karne wali state pe `git commit`, experiment Claude se, ghalat ho to `/rewind`, theek ho to dobara commit.

SEEDHI BAAT

Claude ne ghalat change kar diya? Esc-Esc ya `/rewind`, pichla message chuno, code + conversation restore. Bas ye yaad rakho: Bash se delete/rename hui files wapas nahi aatin.

SESSIONS

08 Sessions — rename, exit, resume, branch

Jab bhi aap kisi folder mein Claude Code kholte ho, ek **session** banta hai. Session = ID + poori history (tool calls samait) + timestamp + metadata + model/permission mode. Claude Code isse continuously aap ki machine pe save karta hai.

Kahan save hota hai (Windows)

```
# PowerShell — apne sessions dekho
dir "$env:USERPROFILE\.claude\projects"

# Har project ka folder (path ke special characters "-" ban jaate hain), andar har session ek file:
C:\Users\<aap>\.claude\projects\C--Users-aap-projects-my-app\<session-id>.jsonl
```

Default retention ~30 din (`cleanupPeriodDays` se badal sakte ho). Ye JSONL files khud edit mat karo — format version ke saath badalta hai; padhna ho to `/export` use karo. `.claude` directory ki poori kahani Video 4 mein.

Session commands

Kaam	Andar se (slash)	Terminal se (PowerShell)
Naam rakhna	`/rename auth-refactor`	`claude -n auth-refactor`
Theek se band karna	`/exit` (alias `/quit`)	—
Last session wapas	—	`claude --continue` (`-c`)
Picker se chunna	`/resume`	`claude --resume` (`-r`)
Naam se resume	`/resume auth-refactor`	`claude --resume auth-refactor`
Branch / copy banana	`/branch try-redis`	`claude --continue --fork-session`
Transcript nikalna	`/export notes.txt`	—

Kyun /exit, terminal close kyun nahi?

`/exit` session ko properly save karta hai aur koi file aadhi likhi (broken state) nahi chhodta. Terminal ka X dabane se agar Claude kuch likh raha tha to file kharab reh sakti hai.

Beginner vs Pro

Beginner

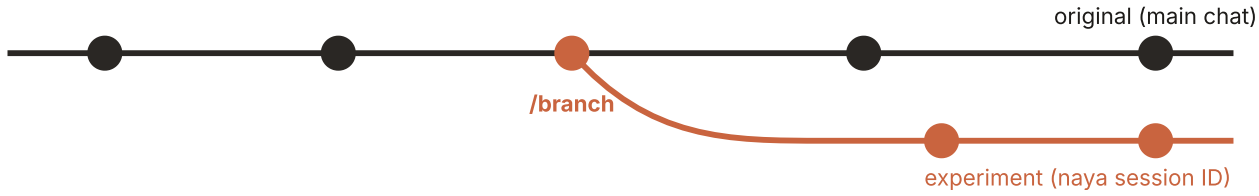
Kal VS Code band kiya, aaj naya session khola: "dark mode toggle add karo". Claude ko project dobara samjhna parta hai — tokens bhi, context bhi zaya.

Pro

`claude --resume task-app` — kal ki poori conversation wapas, Claude ko sab pata hai. Session ka naam feature/ticket pe rakha hua hai.

/branch — experiment bina asli chat kharab kiye

ChatGPT ka "Branch in new chat" yaad hai? Wahi. Aap ki ek acchi chat chal rahi hai, kisi ne kaha "ek naya feature try karke dekho" — lekin main chat disturb nahi karni. `/branch` conversation ki copy bana kar aap ko us mein le jata hai; original disk pe wese hi rehti hai, `/resume` se wapas.



VIDEO SAID → NOW

Video mein `/fork` ko `/branch` jaisa bataya gaya. Ab ye **do alag commands** hain: `/branch` = copy bana ke **usi terminal mein switch**; `/fork` = copy ko alag **background session** ki tarah chalana (agent view). Aur rewind se bhi conversation wapas jati hai, lekin wo usi session mein hota hai — naya session nahi.

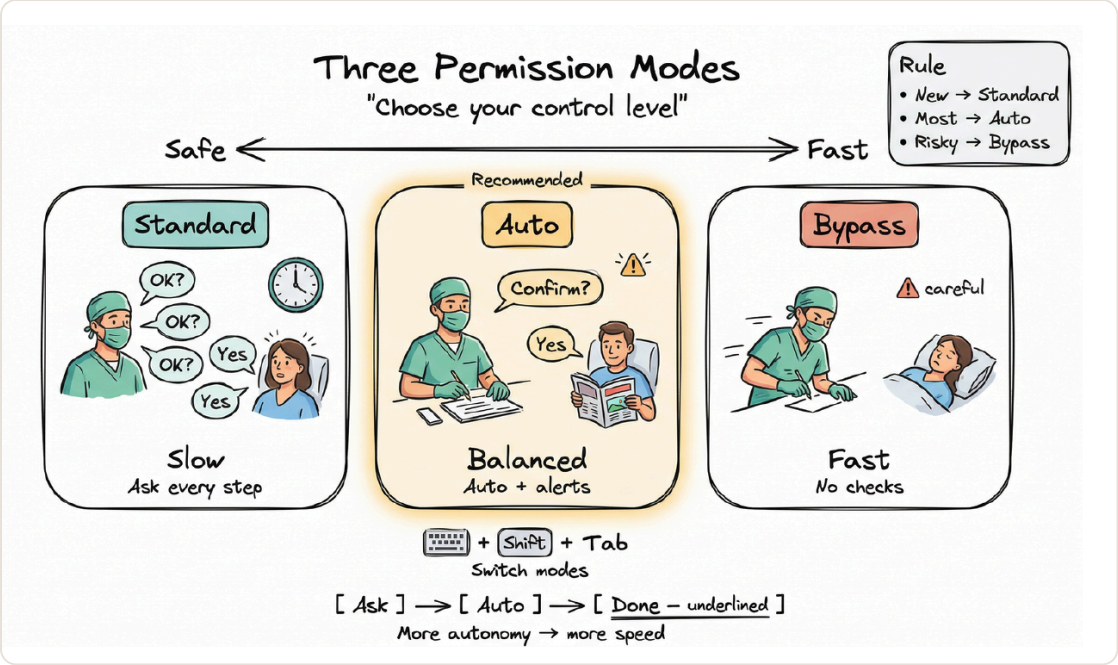
SEEDHI BAAT

Har kaam ka session naam se: `/rename`. Band karna: `/exit`. Kal wapas: `claude -c` ya `claude -r naam`. Kuch try karna hai bina risk: `/branch`.

SAFETY

09 Permission modes

Claude ka **dimagh** sochta hai, phir **haath** kaam karte hain — file banana, delete karna, terminal command chalana. Ye taqatwar hai, lekin Claude ghalti bhi kar sakta hai. **Permission mode** tay karta hai ke Claude bina poochhe kitna kar sakta hai — yaani aap kitni supervision chahte ho. Docs ke mutabiq ye Claude Code ka sab se important safety concept hai, aur sab se zyada ghalat samjha jane wala.



Course board: "Three Permission Modes" — Standard (har step pe poochhe), Auto (balanced), Bypass (koi check nahi)

Manual mode mein har edit pe 3 options aate hain: **Yes**, **Yes, allow all edits during this session**, **No**. Bare project mein har baar "Yes" dabana development slow kar deta hai — modes isi ka hal hain.

Saare modes (official docs table)

Mode	Bina poochhe kya chalta hai	Best for
`default` (UI mein "Manual")	Sirf reads	Har action khud review karna, sensitive kaam
`acceptEdits`	Reads, file edits, aam filesystem commands (`mkdir`, `touch`, `mv`, `cp`)	Jis code ko aap review kar rahe ho us pe iterate karna
`plan`	Reads (+ classifier-approved commands jab auto available ho)	Badalne se pehle codebase explore karna
`auto`	Sab kuch, background safety checks ke saath	Lambe tasks, "yes yes" ki thakan kam karna
`dontAsk`	Sirf pehle se approved tools; jo prompt maangta wo deny	Locked-down CI aur scripts
`bypassPermissions`	Sab kuch (protected paths ke ilawa)	**Sirf** isolated containers / VMs

Shift+Tab — mode badlo



Har Shift+Tab agla mode · status bar mein dikhta hai: "⏵⏵ auto mode on", "⏸ plan mode on" … · bypass sirf launch flag se cycle mein aata hai

VIDEO SAID → NOW (BARA CHANGE)

Video mein instructor ko auto mode Pro plan pe nahi mila ("Team/Enterprise only, research preview, March mein launch"). **Ab:** auto mode **saare plans** pe available hai, aur naye versions mein terminal aur VS Code ka **built-in starting mode hi auto hai** (Pro/Max/Team pe native Windows v2.1.233+ se, baaki sab pe v2.1.283+). Supported model chahiye (Opus 4.6+, Sonnet 4.6+, Haiku 5.5, Fable). `claude --version` se apna version dekho.

Auto mode andar se kya karta hai

Aap ki jagah ek **doosra model (classifier)** har action review karta hai. Kuch cheezein default allow, kuch default block:

Allowed by default

- Working directory mein local file operations
- Lock file / manifest mein declared dependencies install
- Read-only HTTP requests
- `.env` parh ke matching API ko credentials bhejna
- Apni repo ki branches pe push

Blocked by default

- `curl | bash` jaisa code download + run
- Production deploys aur migrations, force push
- Sensitive data bahar bhejna, secrets commit karna
- Session se pehle ki files ko permanently destroy karna
- `git reset --hard`, `git clean -fd`, cloud storage mass delete

Aap ki baat bhi rule hai: agar aap conversation mein kaho "push mat karna" ya "deploy se pehle mujhe dikhana", classifier usko block signal maanta hai — chahe default rules allow karte hon.

CAREFUL — BYPASS / --DANGEROUSLY-SKIP-PERMISSIONS

Video mein accept-edits mode mein "saari .txt files delete karo" bola to Claude ne bina poochhe delete kar di. Bypass mein to **kuch bhi** bina poochhe chalega — ek ghalat command poora folder uda sakti hai. Isko **sirf container/VM** mein use karo, apne Windows laptop pe kabhi nahi. Auto mode bhi "safety guarantee" nahi hai — sensitive kaam pe review khud karo.

Default mode set karna (Windows)

```
# Ek session ke liye — PowerShell
claude --permission-mode plan
claude --permission-mode acceptEdits

# Hamesha ke liye — %USERPROFILE%\.claude\settings.json
{
  "permissions": { "defaultMode": "acceptEdits" }
}
```

Note: `"auto"` aur `"bypassPermissions"` project ki `.claude\settings.json` se apply nahi hote (security ke liye) — auto ko user-level file `%USERPROFILE%\.claude\settings.json` mein rakho.

VS Code mein: `Ctrl + ,` (Settings) → "Claude Code" search karo → **Initial Permission Mode** (aur sidebar/panel location wagera). Video mein Mac ka Cmd+comma tha; Windows pe Ctrl+comma. Chat mein mode indicator pe click karke bhi badal sakte ho.

SEEDHI BAAT

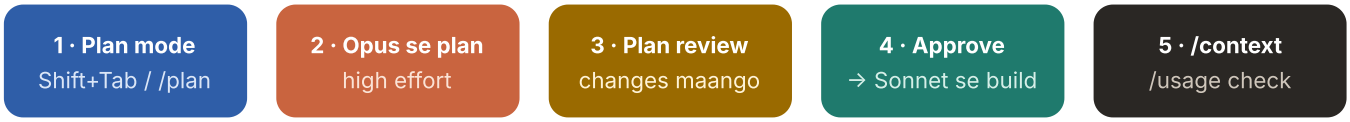
Naya / anjaan project → **plan** ya **manual**. Roz ka kaam → **auto** (ab default). Code khud line-by-line review kar rahe ho → **accept edits**. CI → **dontAsk**. Bypass → sirf container. **Shift+Tab** se badlo.

WORKFLOW

10 Plan mode workflow

Plan mode mein Claude **pehle poora codebase explore karta hai aur plan dikhata hai — edit nahi karta.** Video demo: "Build a complete REST API (get/list/create tasks, error handling)" plan mode mein diya. Claude ne:

1. Directories list ki, files parhi (bilkul jaise insaan karta)
2. `pip list` / version checks chalaye — dekha FastAPI, Pydantic pehle se installed hain — lekin **kuch install nahi kiya**
3. Pichle task-manager project ko pehchana aur usi pe FastAPI backend add karne ka plan banaya (file layout, endpoints, run kaise karna)
4. Plan alag file mein khola — approve, reject ya changes maangne ka option



- Plan mode mein jaane ke tareeqe: **Shift+Tab**, ek prompt ke shuru mein `/plan`, ya `claude --permission-mode plan`.
- Plan pe feedback do: "verification curl se mat karo", "pehle tests likho", ya non-coder ke liye "ye plan simple English mein samjhao" — video mein Claude ne plan ko plain English mein dobara likh diya.
- Plan approve karte hi Claude plan mode se nikal kar edit karna shuru karta hai; session ko plan ke hisaab se title bhi mil jata hai.
- Plan chal raha ho aur rokna ho to **Esc** — lekin jab wo commands chala raha ho beech mein mat roko; research/reading phase mein rokna theek hai.

ENGINEER TIP — "OPUS SE SOCHO, SONNET SE LIKHO"

Har naye ya bare feature ke liye: plan mode + Opus (high effort) → plan ko khud padho aur edit karwao → approve → `/model sonnet` → build. Ya seedha `/model opusplan` rakh do: plan mode mein Opus, execution mein Sonnet automatic. Bara faida: ghalat architecture pe hazaron tokens kharch hone se pehle hi ghalti pakdi jati hai.

SEEDHI BAAT

Bara kaam? Seedha "bana do" mat bolo. Pehle plan mode, plan parho, theek karwao, phir approve. Ye 2 minute 2 ghante bachaate hain.

COMMANDS

11 Baaki useful commands

Command	Kaam	Kab
`/login`	Anthropic account se sign in (account / subscription badalna)	Naya account, API vs subscription switch
`/logout`	Sign out — agli baar `claude` chalao to dobara authorize, security notes wagera, bilkul fresh start	Shared machine, account badalna
`/status`	Version, model, account, connectivity	"Kaunsa model/account chal raha hai?"
`/config`	Settings UI — theme, model, auto-compact, thinking mode, tips	Ek dafa setup
`/powerup`	Claude ke andar interactive animated lessons	Naye features seekhne ke liye
`/help`	Saare commands	Jab yaad na aaye
`/doctor`	Installation diagnose; `f` dabao to auto-fix	Kuch toota lage (Video 2)
`/btw`	Side sawal, main history mein add nahi hota	Kaam ke beech chhota sawal
`/export`	Conversation text file / clipboard mein	Notes, documentation, sharing

/powerup ke lessons (video mein dikhaye gaye)

Talk to your codebase · Steer with modes · Undo anything · Run in the background · Teach Claude your rules (CLAUDE.md) · Extend with tools · Automate your workflow · Multiply yourself (subagents/teams) · Code from anywhere (remote control) · Dial the model. Inme se zyada tar topics course ki agli videos mein aayenge.

> **ENGINEER TIP**
>
> Instructor khud local development VS Code mein aur client work ek server pe terminal se karta hai — isliye slash commands seekhna zaroori hai: server pe koi UI nahi hota, sirf terminal aur commands. Jo aaj terminal mein seekh liya wo har jagah chalega.

FOR AI ENGINEERS

12 AI Engineer playbook

Video 3 ke concepts ko real AI engineering kaam (RAG pipelines, LLM agents, chatbots, FastAPI backends, eval scripts) pe kaise lagana hai:

Ek feature ka poora daily flow (PowerShell)

```
# 1. Project folder mein, naam ke saath session
cd C:\projects\rag-service
claude -n add-reranker

# 2. Andar: plan mode + Opus
/model opusplan          # ya /model opus
#    Shift+Tab → plan mode
"Retriever ke baad cross-encoder reranker add karna hai. Existing pipeline padho, plan do."

# 3. Plan review → approve → Sonnet build karta hai
# 4. Ghalat direction? Esc Esc → /rewind
# 5. Lambi ho gayi? /compact sirf reranker design aur open bugs yaad rakho
# 6. Kaam khatam: /usage → git commit → /exit

# Agle din
claude -r add-reranker
```

Situation → command

AI engineering situation	Kya karo
Naya LLM agent / RAG service architecture	Plan mode + Opus + high effort; plan approve karke Sonnet
Prompt template ya chain pe A/B experiment	`/branch prompt-v2` — main session safe
Embedding model badalne ka experiment fail hua	`/rewind` → restore code and conversation (aur Git)
Bari eval logs / stack traces paste karte karte context bhar gaya	`/context` dekho → `/compact` focus ke saath, ya log files ko path se refer karo, paste mat karo
Backend ka kaam khatam, ab frontend	`/clear` (naya topic = naya context)
Lint / typo / chhoti fix	`/model haiku`, low effort
Bara refactor ya test generation ke baad	`/usage` — limit % check
Production DB / deploy scripts wala repo	Manual ya plan mode; chat mein likho "deploy/migrate mat karna"
Kai features parallel	Har feature ka named session; `/resume` picker se switch

CAREFUL — SECRETS

AI projects mein `.env` mein API keys hoti hain (OpenAI, Anthropic, vector DB). Auto mode `.env` parh kar matching API ko key bhej sakta hai (allowed), lekin secrets ko commit/push karna ya bahar bhejna block hai. Phir bhi: `.env` ko `.gitignore` mein rakho, aur keys kabhi chat mein paste mat karo — har message ke saath wo context mein dobara jaati hain aur session file mein disk pe save hoti hain.

ENGINEER TIP — CONTEXT = PAISA, KHAAS KAR AGENTS MEIN

Jo aap ne is video mein seekha (har turn pe poori history dobara jati hai, cache, compaction) wahi cheez aap ke apne LLM apps pe bhi lagti hai jab aap Claude API pe chatbot ya agent banate ho. Claude Code ka `/context` grid dekhna aap ko apne agents ke token budget design karna bhi sikhata hai: system prompt chhota, sirf zaroori tools, history summarize.

VERIFIED · 21 OCT 2026

13 Video said → Now

Ye video kuch mahine purani hai. Claude Code docs (code.claude.com) se check karne pe ye farq mile:

#	Video mein	Ab (current docs)
1	Default model Sonnet 4.6; Opus 4.6; Haiku 200K context	`default` = Opus 5.5; `sonnet` = Sonnet 5.5; `haiku` = Haiku 5.5; current models pe 1M context default
2	Auto mode Pro plan pe nahi; sirf Team/Enterprise; research preview	Auto mode saare plans pe; v2.1.283+ mein built-in starting mode hi auto hai
3	`/cost` aur usage alag commands	`/usage` asal; `/cost` aur `/stats` aliases
4	`/fork` = `/branch`	Alag: `/branch` = copy + switch; `/fork` = copy as background session
5	Auto-compact ~170–180K pe	1M models pe ~967K pe — manual `/compact` ab aur important
6	Rewind ke baad `database_config` "kisi problem" se reh gaya	Mumkin wajah: Bash (`mv`) se hui changes checkpoint mein track nahi hotin
7	Default mode "Ask before edits" / "normal"	Ab iska naam **Manual** hai (config value `default`)
8	Effort: low / medium / high / max	low, medium, high, **xhigh**, max, auto; 5.5 models pe default medium; `max` sirf session-only
9	VS Code settings: Cmd + comma	Windows: **Ctrl + comma**
10	`/clear` current session saaf karta tha	Nayi conversation shuru; purani save rehti hai (`/resume` se wapas)

SEEDHI BAAT

Concepts sab same hain (context, sessions, modes) — sirf naam, numbers aur defaults update hue. Jab bhi kuch alag dikhe: `/help` , `/release-notes` , ya docs.

REVISION

14 Cheat sheet

Model

`/model` brain · `/effort` soch ki gehrai · `/fast` tez output

Opus plan → Sonnet build → Haiku maintain · beech mein switch kam

Cost

`/usage` (= `/cost`, `/stats`) · `/status`

Token = currency · output tokens mehnge

Context

`/context` dekho · `/compact [focus]` chhota karo · `/clear` naya topic · `/btw` side sawal

Har message pe system + tools + MCP + memory + poori history jati hai

Undo

`/rewind` ya **Esc Esc** · code / conversation / dono / summarize

Bash changes track nahi · Git replacement nahi

Sessions

`/rename` · `/exit` · `/resume` · `/branch`

`claude -n naam` · `claude -c` · `claude -r naam`

%USERPROFILE%\.claude\projects · ~30 din

Permissions

Shift+Tab: auto → manual → accept edits → plan

`--permission-mode plan` · `dontAsk` CI · bypass sirf container

Khud se poochho (self-test)

1. "hi" bhejne pe bhi ~29K tokens context kyun dikhta hai?
2. `/compact` aur `/clear` mein kya farq hai, aur kab kaunsa?
3. Rewind ke baad ek file wapas kyun nahi aayi? Isse bachne ke liye kya karoge?
4. Beech conversation mein model badalna mehnga kyun hai?
5. `/branch` aur `/rewind` dono "pichle point" pe le jaate hain — farq kya hai?
6. Production wale repo pe kaunsa mode, aur auto mode ko kaise bataoge ke deploy na kare?
7. Windows pe aap ke sessions disk pe kahan hain, aur unhe naam se kaise khologe?

AGLA VIDEO (4)

The `.claude` directory: CLAUDE.md (project ki "new hire brief"), rules, settings.json mein permission rules, skills, agents — wahan permission modes aur commands ko file ke zariye permanent banana seekhenge.

COURSE RESOURCE

A All slash commands — 74 commands, 9 categories

Source: course repo ki `claude-slash-commands-guide.html` (instructor ne Claude se banwayi). Kuch entries ab badal chuki hain — chapter 13 dekho (jaise `/cost` aur `/stats` ab `/usage` ke aliases hain, `/fork` ab alag command hai). Aap ke version mein jo commands hain wo `/` type karke dekho.

Getting Started & Help 6

Command	Kaam
`/help`	Show all available commands and how to use them.
`/init`	Initialize your project with a CLAUDE.md guide — the memory file Claude reads at the start of every session.
`/powerup`	Discover Claude Code features through quick interactive animated lessons.
`/doctor`	Diagnose and verify your Claude Code installation and settings. Press f to auto-fix any issues found.
`/release-notes`	View the changelog in an interactive version picker to see what changed in each release.
`/feedback`	Submit feedback or bug reports directly to Anthropic. Alias: /bug

Session Management 8

Command	Kaam
`/clear`	Wipe the conversation history and start completely fresh. Aliases: /reset, /new
`/compact [instructions]`	Summarize older messages to free up context window space while keeping the important parts.
`/branch [name]`	Create a fork of the current conversation at this point. Alias: /fork
`/resume [session]`	Resume a past conversation by ID or name, or open a session picker. Alias: /continue
`/rewind`	Rewind the conversation and/or your code to a previous point in history. Alias: /checkpoint
`/export [filename]`	Export the current conversation as plain text to clipboard or a file.
`/rename [name]`	Give the current session a meaningful name shown on the prompt bar.
`/exit`	Exit the Claude Code CLI cleanly. Alias: /quit

Memory & Context 4

Command	Kaam
`/memory`	Edit CLAUDE.md memory files, enable/disable auto-memory, and view auto-memory entries.

`/context`	Visualize how much of your context window is used, shown as a colored grid with optimization suggestions.
`/add-dir <path>`	Add an extra working directory so Claude can read files from it during this session.
`/btw <question>`	Ask a quick side question without adding it to the main conversation history.

Model & Configuration 7

Command	Kaam
`/model [model]`	Switch the AI model Claude uses. Use arrow keys to adjust effort level on supported models.
`/effort [low|medium|high|max|auto]`	Set how hard Claude thinks — from quick responses to maximum extended thinking. Max requires Opus.
`/config`	Open the Settings interface to adjust theme, model, output style, and preferences. Alias: /settings
`/theme`	Change the color theme. Includes light, dark, colorblind-accessible, and ANSI themes.
`/color [color|default]`	Set the prompt bar color for the current session (red, blue, green, yellow, purple, orange, pink, cyan).
`/fast [on|off]`	Toggle fast mode — prioritizes speed over thoroughness.
`/sandbox`	Toggle sandbox mode — a safe, isolated environment for running code. Available on supported platforms.

Code & Development 9

Command	Kaam
`/plan [description]`	Enter plan mode — Claude discusses and maps out a strategy before writing any code.
`/ultraplan <prompt>`	Draft a deep plan in an ultraplan session, review it in your browser, then execute remotely or send it back to your terminal.
`/batch <instruction>`	Orchestrate large-scale codebase changes by spawning multiple parallel agents in isolated git worktrees.
`/autofix-pr [prompt]`	Spawn a cloud session that watches your PR and automatically pushes fixes when CI fails or reviewers comment.
`/security-review`	Analyze pending changes on the current branch for security vulnerabilities such as injection, auth issues, and data exposure.
`/simplify [focus]`	Review recently changed files for code quality, reuse, and efficiency — then apply fixes automatically.
`/debug [description]`	Enable debug logging for the current session and troubleshoot issues by reading the session debug log.
`/diff`	Open an interactive diff viewer showing uncommitted changes and per-turn diffs from Claude.
`/claude-api`	Load Claude API reference material for your project's language. Also auto-activates when your code imports the Anthropic SDK.

Integrations 11

Command	Kaam
`/mcp`	Manage MCP (Model Context Protocol) server connections and OAuth authentication.
`/install-github-app`	Set up the Claude GitHub Actions app to automatically review your pull requests.
`/install-slack-app`	Install the Claude Slack app via OAuth to interact with Claude from Slack.
`/ide`	Manage IDE integrations such as VS Code and JetBrains, and show their connection status.
`/chrome`	Configure Claude in Chrome — the browser automation integration settings.
`/desktop`	Continue the current terminal session in the Claude Code Desktop app. macOS and Windows only. Alias: /app
`/voice`	Toggle push-to-talk voice dictation — speak your prompts instead of typing. Requires Claude.ai account.
`/remote-control`	Make this terminal session accessible for remote control from claude.ai. Alias: /rc
`/teleport`	Pull a Claude Code on the web session into your local terminal. Alias: /tp
`/web-setup`	Connect your GitHub account to Claude Code on the web using your local gh CLI credentials.
`/mobile`	Show a QR code to download the Claude mobile app. Aliases: /ios, /android

Account & Usage 10

Command	Kaam
`/login`	Sign in to your Anthropic account.
`/logout`	Sign out from your Anthropic account.
`/cost`	Show token usage statistics for the current session.
`/usage`	Show your plan usage limits and current rate limit status.
`/stats`	Visualize daily usage, session history, streaks, and model preferences.
`/insights`	Generate a report analyzing your Claude Code sessions: project areas, interaction patterns, and friction points.
`/upgrade`	Open the upgrade page to switch to a higher plan tier. Only visible to eligible users.
`/extra-usage`	Configure extra usage to keep working when rate limits are hit.
`/privacy-settings`	View and update your privacy settings. Only available on Pro and Max plans.
`/passes`	Share a free week of Claude Code with friends. Only visible if your account is eligible.

Agents, Plugins & Automation 9

Command	Kaam
`/agents`	Manage sub-agent configurations — specialized agents Claude can delegate tasks to.

`/plugin`	Manage Claude Code plugins — packaged bundles of commands, skills, and configurations from the plugin marketplace.
`/reload-plugins`	Reload all active plugins to apply changes without restarting Claude Code.
`/skills`	List all available skills — stored prompts and procedures Claude can use.
`/hooks`	View hook configurations — automated actions triggered by tool events such as run Prettier after every file edit.
`/permissions`	Manage allow/ask/deny rules for what tools Claude can use. Alias: /allowed-tools
`/loop [interval] [prompt]`	Run a prompt repeatedly on a schedule while the session stays open. Alias: /proactive
`/schedule [description]`	Create, update, list, or run routines — persistent scheduled tasks that run even when Claude Code is closed.
`/tasks`	List and manage background tasks currently running. Also available as /bashes

Utilities & Productivity 10

Command	Kaam
`/copy [N]`	Copy the last assistant response (or Nth-last) to clipboard. Shows a picker when code blocks are present.
`/status`	Open the Settings interface (Status tab) showing version, model, account, and connectivity — works mid-response.
`/statusline`	Configure Claude Code's status line — the info bar at the bottom of the terminal.
`/keybindings`	Open or create your keybindings configuration file to define custom shortcuts.
`/terminal-setup`	Configure terminal keybindings such as Shift+Enter for terminals that need special setup like VS Code, Alacritty, and Warp.
`/team-onboarding`	Generate an onboarding guide for teammates from your real Claude Code usage history from the last 30 days.
`/setup-bedrock`	Configure Amazon Bedrock authentication, region, and model settings via interactive wizard.
`/setup-vertex`	Configure Google Vertex AI authentication, project, region, and model settings via interactive wizard.
`/remote-env`	Configure the default remote environment for web sessions started with --remote.
`/stickers`	Order Claude Code stickers.

CLAUDE CODE CRASH COURSE · VIDEO 4 OF 5

The .claude Directory

Project ka control room — CLAUDE.md, CLAUDE.local.md, settings.json, settings.local.json, rules/ aur .mcp.json. Levels, precedence, aur ek AI engineer ka real setup.

```
PS C:\Projects\ai-bot> tree
├── CLAUDE.md
├── CLAUDE.local.md # gitignored
├── .mcp.json
└── .claude\
├── settings.json
├── settings.local.json
└── rules\ skills\ agents\
```

Notes for AI engineers · Windows-focused · Verified against official docs (code.claude.com/docs) on 21 Oct 2026
Source: Mayank's YouTube crash course + course GitHub repo (ClaudeCode/.claude demo files)

CONTENTS

Is PDF mein kya hai

01	.claude folder kya hai — aur Windows par "hidden" ka sach
02	3 levels: Enterprise (Managed) → Global (User) → Project
03	Poora map: project root, .claude\, aur %USERPROFILE%\.claude
04	CLAUDE.md — Claude ka briefing document
05	CLAUDE.local.md — sirf aap ke liye notes
06	settings.json — permissions, env, model (enforced!)
07	settings.local.json — personal overrides
08	rules\ — CLAUDE.md ko tukron mein baanto
09	.mcp.json, skills\, agents\ — quick intro
10	Sabse important concept: Context vs Enforcement
11	AI Engineer ka real .claude setup (LLM / RAG project)
12	Windows par setup — step by step commands
13	Video said → Docs say (corrections)
14	Revision cheat-sheet + practice tasks

SEEDHI BAAT — POORI VIDEO EK LINE MEIN

`.claude` folder woh jagah hai jahan aap Claude ko batate ho **kaise kaam karna hai** (CLAUDE.md, rules) aur **kya kar sakta hai / kya nahi** (settings.json). Yeh project, global aur enterprise — teen levels par hota hai, aur jo cheez project ke jitni qareeb ho, utni zyada strong.

SERIES CONTEXT

Video 1: Claude Code kya hai · Video 2: Install & setup · Video 3: Slash commands, sessions, permission modes · **Video 4 (yeh): .claude directory** · Video 5: CLAUDE.md in depth (next).

CHAPTER 01

.claude folder kya hai?

Har serious Claude Code project mein ek `.claude` folder hota hai. Zyada tutorials sirf `CLAUDE.md` dikhate hain, lekin asal control yahan hai: **agents, skills, rules, permissions, hooks, MCP, settings** — sab isi folder (aur iske aas paas) define hote hain. Claude Code har session start par yeh files khud parhta hai.



Left (orange) = Claude ko context/instructions. Right (green) = Claude Code ki enforced configuration.

Dot (.) se shuru hone wale folders

Video mein bataya gaya ke naam ke shuru mein dot ho to folder "hidden" hota hai, is liye aam log isay dekhte hi nahi. Yeh baat Linux/Mac par sahi hai — Windows par thori different hai:

VIDEO SAID → WINDOWS REALITY

Windows par **dot se naam hidden nahi hota**. Hidden hona ek *file attribute* hai. Jab aap `mkdir .claude` chalate ho, File Explorer aur PowerShell ka `ls` dono isay normally dikhate hain. Sirf woh folders chhupte hain jin par Hidden attribute laga ho (jaise Git ka `.git`).

Kaam (Windows)	Kaise
Hidden items Explorer mein dikhana	File Explorer → **View** → **Show** → **Hidden items** ✓
PowerShell mein sab (hidden bhi) dikhana	`Get-ChildItem -Force` (short: `ls -Force`)
CMD mein	`dir /a`
Global .claude folder kholna	`explorer $env:USERPROFILE\.claude`

SEEDHI BAAT

.claude folder aam user ke liye "chhupa hua" hai kyun ke usay zaroorat nahi. Professional ke liye yahi asli jagah hai jahan Claude ko apni marzi ka banaya jata hai.

CHAPTER 02

3 Levels — kaun kis ko override karta hai

Claude Code configuration teen levels par kaam karti hai. Video ka main rule: **jo project ke jitna qareeb, utni zyada priority** — sirf ek exception: enterprise ki *managed settings* (security rules) ko koi override nahi kar sakta.



Instructions ke liye: Project > Global. Security settings ke liye: Managed sab se upar.

Video ka demo (claude.ai par same idea)

- **Global preference** (claude.ai Settings): "Give me code in Python."
- **Project instruction**: "I work and learn DSA in C++."
- Sawal: "Bubble sort code do" → Claude ne **conflict pakra aur pucha** — kaun sa follow karun?
- Phir chat mein kaha "Give me all code in Java" → ab Java. **Sab se qareeb instruction (current chat) jeet gaya.**

ENGINEER TIP — "WINS" KA MATLAB SAMJHO

Do alag systems hain, aur dono alag tarah se combine hote hain:
• **CLAUDE.md files merge nahi hoti, sab ek saath load hoti hain** (concatenate). Conflict ho to project wali instruction ko priority milti hai — lekin Claude kabhi clarify bhi karta hai (jaise demo mein).
• **settings.json** mein: scalar values (jaise `model`) mein most-specific jeet-ta hai; arrays (jaise `permissions.allow`) **sab levels se combine** ho jaate hain.

Settings ka exact precedence (docs)

#	Level	File	Kis par lagta
1	Managed	`C:\Program Files\ClaudeCode\managed-settings.json` / MDM / claude.ai console	Poori organization
2	Command line	`claude --settings ...`, `--permission-mode`	Sirf yeh session
3	Project local	`.claude\settings.local.json`	Aap, is project mein
4	Shared project	`.claude\settings.json`	Poori team
5	User (global)	`%USERPROFILE%\.claude\settings.json`	Aap, har project

VERIFIED

Kaunsi settings files load hui — check karne ke liye Claude Code ke andar `/status` chalao; "Setting sources" line batati hai. Errors ke liye terminal mein `claude doctor`.

CHAPTER 03

Poora map — kya kahan rehta hai

Video mein official docs ka page "Explore the .claude directory" dikhaya gaya — yeh sab se reliable source hai. Neeche usi ka verified summary hai.

Project (repo ke andar)

```
C:\Projects\my-app\
├── CLAUDE.md           # team instructions
├── CLAUDE.local.md     # personal
(gitignore)
├── .mcp.json           # team MCP servers
└── .claude\
    ├── settings.json        # team
    ├── settings.local.json  # personal
    ├── rules\*.md
    ├── skills\<name>\SKILL.md
    ├── agents\*.md
    ├── commands\*.md   # old style
    ├── output-styles\*.md
    └── agent-memory\
```

Global (%USERPROFILE%)

```
C:\Users\<you>\
├── .claude.json    # app state, login,
│                   # personal MCP
└── .claude\
    ├── CLAUDE.md        # all projects
    ├── settings.json    # defaults
    ├── keybindings.json
    ├── themes\*.json
    ├── rules\ skills\ agents\
    ├── projects\<proj>\memory\
    │       # auto memory
    └── plugins\
```

File	Git commit?	Kya karti hai	Kab load
`CLAUDE.md`	✔ Yes	Project conventions, commands, architecture	Har session. Root ya `.claude\` dono jagah chalta
`CLAUDE.local.md`	✘ No	Aap ki private project notes	CLAUDE.md ke saath, uske baad
`.mcp.json`	✔ Yes	Team ke MCP servers	Session start. **Root par, .claude mein nahi**
`settings.json`	✔ Yes	Permissions, hooks, model, env	Directly applied — **enforced**
`settings.local.json`	✘ No	Personal overrides, same schema	settings.json ke upar
`rules\*.md`	✔ Yes	Topic-wise instructions	Start par; `paths:` ho to matching file par
`skills\`	✔ Yes	Reusable workflows, `/name`	Jab invoke ho
`agents\*.md`	✔ Yes	Subagents (apna context window)	Jab invoke ho

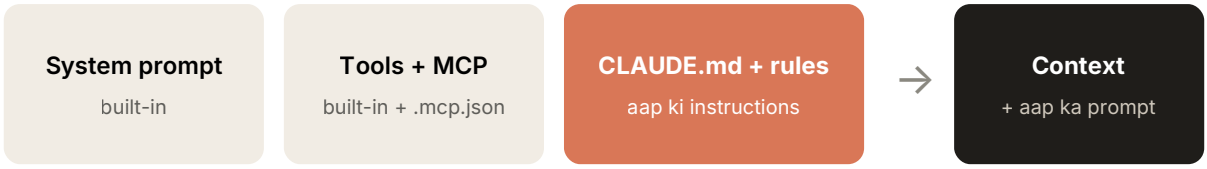
SEEDHI BAAT

Jo team ke saath share karna hai → normal file (commit). Jo sirf aap ka hai → `.local` wali file (commit mat karo). Jo har project mein chahiye → global `%USERPROFILE%\.claude`.

CHAPTER 04

CLAUDE.md — Claude ka briefing document

Claude ke paas pehle se system instructions, tools aur MCP hote hain. **CLAUDE.md inke saath context mein add ho jaati hai** — har conversation ke start par. Socho ek naye developer ko pehle din project ka briefing doc dete ho: stack, structure, conventions, kya karna hai kya nahi.



Video ka demo CLAUDE.md (repo se, short)

```
# Project: Task Manager App
## What This Is
A web-based task management app. Stack: HTML, CSS, vanilla JS. No frameworks.
## Project Structure
- index.html — main app shell · style.css — all styling · app.js — all JS
## Coding Standards
- camelCase for JS variables/functions · kebab-case for CSS classes
- Every function must have a JSDoc comment
## Behaviour Rules
- Never use alert() or confirm() — use custom modal components
## Git Convention
- Commit messages: "[type]: description" — e.g. "feat: add priority labels"
- Never commit directly to main
```

Result: naye chat mein "Write a JS function to filter tasks" → Claude ne **pehle CLAUDE.md parhi** aur camelCase + JSDoc follow kiya, bina bataye.

Verified best practices

- **Size: 200 lines se kam.** Lambi file = zyada context kharch + kam adherence. (Video: "100–200 lines target".)
- **Location:** `.\CLAUDE.md` ya `.\.claude\CLAUDE.md` — dono valid. Root clean rakhna ho to .claude ke andar.
- **`/init`** — codebase scan kar ke starter CLAUDE.md bana deta hai (pehle se ho to improvements suggest karta hai).
- **Imports:** `@docs/api.md` likhne se woh file bhi load hoti hai (max 4 hops). Lekin imports context kam nahi karte — woh bhi start par load hote hain.
- **Loading:** working folder aur uske upar ke saare CLAUDE.md load; **sub-folders ke CLAUDE.md lazy** — jab Claude us folder ki file chhoye.
- Jo cheez sirf kisi ek task par chahiye → **skill** ya **path-scoped rule** mein daalo, CLAUDE.md mein nahi.

> **ENGINEER TIP**
>
> CLAUDE.md mein woh likho jo Claude khud code dekh kar **nahi** samajh sakta: "tests ke liye `pytest -q` chalao", "prompts sirf `prompts\` folder mein", "OpenAI direct call mat karo, `llm_client.py` wrapper use karo". Detail ke liye Video 5 (CLAUDE.md in depth).

CHAPTER 05

CLAUDE.local.md — sirf aap ke liye

Same kaam jo CLAUDE.md karti hai, lekin **personal** — aap ki machine ki details, aap ki preferences. Yeh GitHub par share nahi honi chahiye.

CLAUDE.md (team)

- Stack, structure
- Coding standards
- Git conventions

+

CLAUDE.local.md (only you)

- "Main Windows 11 par hoon"
- Local port 9000 (prod 8000)
- Sandbox URL, test data

Dono load hote hain. Same folder mein CLAUDE.local.md, CLAUDE.md ke **baad** parhi jaati hai.

```
# Local Overrides (not committed to git)
## My Local Setup
- I'm on Windows 11, PowerShell 7. Python: py -3.12
- Local API runs on http://localhost:9000 (production uses 8000)
- My API keys are in .env (never print or commit this)
## Personal Preferences
- Verbose error messages during development
```

VIDEO SAID → DOCS SAY (2 CORRECTIONS)

1. Location: Video/repo mein yeh `.claude\CLAUDE.local.md` mein rakhi gayi. Docs ke mutabiq iski jagah **project root** hai: `.\CLAUDE.local.md`. (CLAUDE.md ke barkhilaf, docs iska `.claude\` wala variant list nahi karti — root par rakho, safe.)
2. Gitignore automatic nahi: Video: "GitHub .local files ignore kar deta hai." Asal mein Git khud kuch ignore nahi karta. Docs: CLAUDE.local.md **aap ko khud `.gitignore` mein add karni hai**, aur yeh Claude Code khud nahi banata — aap manually banao.

```
# PowerShell — project root mein:
New-Item -ItemType File CLAUDE.local.md
Add-Content .gitignore "CLAUDE.local.md"
```

SEEDHI BAAT

CLAUDE.md = office ka rulebook (sab ke liye). CLAUDE.local.md = aap ki apni diary (sirf aap ke liye) — isko .gitignore mein zaroor daalo.

CHAPTER 06

settings.json — yeh request nahi, rule hai

CLAUDE.md Claude ko *batati* hai. `settings.json` Claude Code ko *configure* karti hai — permissions, hooks, model, env variables, status line, output style. **Yeh enforced hai: Claude chahe ya na chahe, deny wali cheez nahi chalegi.**

Video ka demo (repo file)

```
{
  "permissions": {
    "allow": ["Bash(git commit*)", "Bash(git add*)", "Bash(npm install*)",
              "Bash(python -m pytest*)", "Write(.claude/*)"],
    "deny":  ["Bash(rm -rf*)", "Bash(rm)", "Bash(sudo*)", "Bash(curl* | bash*)"]
  },
  "env": { "NODE_ENV": "development", "DEBUG": "true" }
}
```

Demo: Claude se kaha "utils folder recursively delete karo" → jawab: **"Permission to use Bash with command rm -rf ... has been denied."** Pehle yeh har baar puchta tha; ab rule ne seedha rok diya.

Permission rules — 3 lists



Isi order mein check hota hai — pehla match decide karta hai. Allow rule kabhi deny ko kaat nahi sakta.

Syntax jo yaad rakhna hai (verified)

Rule	Matlab
`Bash(npm run test *)`	`npm run test` + koi bhi arguments (bare command bhi)
`Bash(ls *)` vs `Bash(ls*)`	**Space matter karta hai.** Pehla `lsof` match nahi karta; doosra karta hai
`Bash(git log:*)`	Purana style, `Bash(git log *)` jaisa hi
`Read(./.env)`	Claude .env file nahi parh sakta (deny mein daalo)
`PowerShell(Remove-Item *)`	**Windows:** PowerShell commands ke rules — aliases (`rm`, `del`) bhi cover
`WebFetch(domain:docs.python.org)`	Sirf is domain ka fetch

- **Compound commands:** `cd x && rm -rf y` mein har sub-command alag check hota hai — deny rule kisi bhi hisse par lag jata hai.
- **Lists merge hoti hain:** global + project + local ke allow/deny sab combine.
- `/permissions` se UI mein rules dekho/add karo — har rule ki source file bhi dikhti hai.

IMPORTANT LIMIT — BASH DENY EK "SECURITY WALL" NAHI

Docs khud kehti hain: `Bash(rm *)` sirf usi form ko rokta hai jo Claude normally likhta hai. `/bin/rm -rf` ya `bash -c 'rm -rf ...'` jaise forms is se bach sakte hain. Asal hard-blocking ke liye **PreToolUse hook** ya **sandbox** (aage ki videos) use karo. Deny = mazboot guard-rail, fortress nahi.

Windows ke liye behtar version of demo settings.json

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "allow": [
      "Bash(git status)", "Bash(git diff *)", "Bash(git add *)", "Bash(git commit *)",
      "Bash(python -m pytest *)", "Bash(pip install -r requirements.txt)"
    ],
    "ask":   [ "Bash(git push *)" ],
    "deny":  [
      "Bash(rm -rf *)", "Bash(rm -r *)",
      "PowerShell(Remove-Item *)",
      "Read(./.env)", "Read(./.env.*)", "Read(./secrets/**)"
    ]
  },
  "env": { "PYTHONUTF8": "1" }
}
```

VERIFIED DETAILS

- Settings files **strict JSON** hain — `//` comment ya trailing comma = Settings Error.
- `$schema` line VS Code mein autocomplete + validation deti hai.
- Team ki committed file mein `allow` rules (aur zyada tar `env`) tab apply hote hain jab folder ko **trust** karo; `deny` / `ask` foran apply.
- Docs ka official example `npm test` / `npm run` allow, `rm -rf` deny, aur har edit ke baad Prettier chalane wala **hook** dikhata hai — hooks ek alag video mein.

Baqi useful keys (ek nazar)

Key	Kaam	Example
`model`	Default model is project ke liye	`"model": "claude-sonnet-5-5"`
`env`	Har session mein env variables	`"env": {"DEBUG": "true"}`
`hooks`	Events par commands auto-run	PostToolUse → formatter
`outputStyle`	Jawab ka style	—
`statusLine`	Terminal ki bottom line	—
`enabledMcpjsonServers`	.mcp.json ke kaunse servers on	`["tavily-remote-mcp"]`

SEEDHI BAAT

Pehle Claude har command par puchta tha (Video 3 ke permission modes). settings.json se aap pehle hi bata do: "yeh hamesha chala lo, yeh kabhi nahi". Jitna aap allow/deny smart banao ge, utni speed aur safety dono.

CHAPTER 07

settings.local.json — personal overrides

Same schema jaise settings.json, lekin sirf aap ke liye, is project mein. Team ki file sab ke liye `"model": "claude-sonnet-5"` kehti ho aur aap ko Opus chahiye → apni local file mein daalo, team par asar nahi.

- **Claude Code khud bhi likhta hai:** jab permission prompt par "**Yes, and don't ask again**" chuno, woh rule yahan `allow` mein save hota hai. (Video demo: `git init` allow ke baad local file mein aa gaya.)
- **Git se bahar:** jab Claude Code pehli dafa yeh file likhta hai to isay global git excludes mein add kar deta hai. **Agar aap ne haath se banayi**, to `.gitignore` mein khud add karo.
- Local file ke allow rules ko trust step nahi chahiye (jab tak file untracked hai).

```
{
  "model": "claude-opus-5-5",
  "env": { "LLM_BASE_URL": "http://localhost:9000" },
  "permissions": { "allow": ["Bash(git init)"] }
}
```

ENGINEER TIP — SECRETS

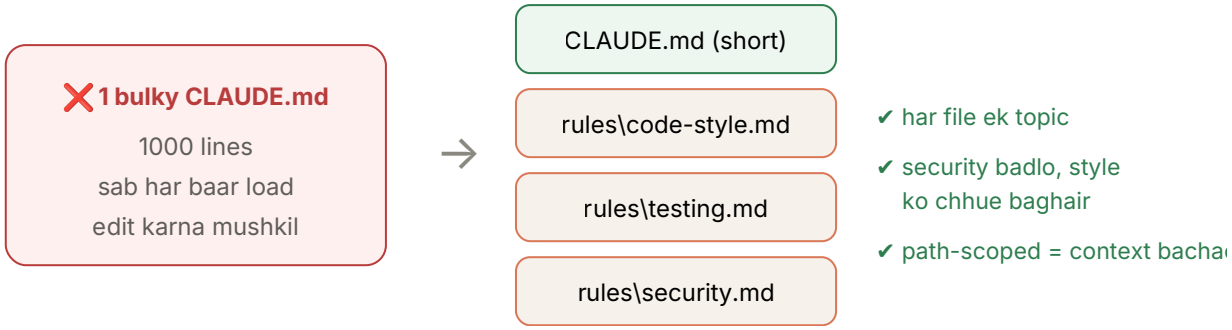
Video ne kaha "local par apna OpenAI key env mein rakho". Behtar: **API keys kisi bhi settings file mein mat likho** — Windows user environment variable ya `.env` (gitignored + `Read(./.env)` deny) use karo. Settings mein sirf non-secret values (URLs, flags). Yaad rahe: transcripts `%USERPROFILE%\.claude` mein plaintext save hote hain.

	settings.json	settings.local.json
Kis ke liye	Poori team	Sirf aap
Git	Commit karo	Kabhi nahi
Priority	Neeche	Upar (overrides)
Kaun likhta	Aap / team lead	Aap + "don't ask again" approvals

CHAPTER 08

rules\ — CLAUDE.md ko tukron mein baanto

Sawal: sab kuch ek CLAUDE.md mein kyun na likh dein? Jawab: **1000 line ki CLAUDE.md kaam nahi karti** — har session poori load hoti hai, context bhar jata hai, aur zyada context = Claude ka focus kharab (Video 3 ka context lesson). Is liye topic-wise files:



Video ke demo rules (repo se)

```
# rules\code-style.md
- camelCase for JS variables
- kebab-case for CSS classes
- snake_case for Python
- All functions must have
  docstrings or JSDoc
- Max line length: 100
```

```
# rules\security.md
- Never hardcode credentials,
  API keys, or passwords
- Use environment variables
- Sanitise all user inputs
- Never use eval() in JS
- Parameterise DB queries
```

Demo result: "Python file banao jo do numbers add kare" → is baar function mein **docstring** aa gayi (pehle wali files mein nahi thi). Phir purani `number_facts.py` ke liye bhi Claude ne khud kaha: "file already exists — project ke coding standard ke mutabiq docstrings add kar deta hoon." Standard kahan se aaya? `code-style.md` se.

VIDEO/REPO → DOCS: FRONTMATTER

Repo ki rule files `description:` frontmatter use karti hain. Docs: rules mein **sirf `paths`** field parha jata hai — baqi sab chupchaap ignore. Rule phir bhi kaam karta hai (bina paths = har session load), bas `description` ka koi asar nahi.

Path-scoped rules — pro level (video mein nahi tha)

Rule sirf tab load ho jab Claude matching file chhoye. Context bachta hai.

```
# .claude\rules\prompts.md
---
paths:
  - "prompts/**/*.md"
  - "src/llm/**/*.py"
---
# Prompt & LLM code rules
- System prompts live in prompts/, never inline strings in Python
- Every prompt change needs an eval run: python -m evals.run --suite smoke
- Always set temperature and max_tokens explicitly
- Parse model output with Pydantic; never trust raw JSON
```

Pattern	Match karta hai
`**/*.py`	Har folder ki Python files
`src/**/*`	src ke andar sab
`src/**/*.{ts,tsx}`	Brace expansion — dono extensions
`tests/**/test_*.py`	pytest files

- Rules recursively discover hote hain — `rules\backend\`, `rules\frontend\` sub-folders bana sakte ho.
- Global rules: `%USERPROFILE%\.claude\rules\` — har project par.
- **Rules vs Skills:** rules hamesha (ya matching file par) context mein; skills sirf jab task ho. "Har baar follow karo" → rule. "Kabhi kabhi ka workflow" → skill.

> **SEEDHI BAAT**
>
> Non-tech misaal: copywriter ko article likhwana hai — ek page "style guide", ek "fact-check rules", ek "legal do's & don'ts". Ek mota register nahi. Yahi rules\ folder hai.

CHAPTER 09

.mcp.json, skills\, agents\ — intro

Video ne inhe sirf touch kiya; detail aage ki videos mein. Lekin location sahi yaad rakho:

.mcp.json — team ke MCP servers

```
# project ROOT par: C:\Projects\my-app\.mcp.json
{
  "mcpServers": {
    "tavily-remote-mcp": {
      "type": "http",
      "url": "https://mcp.tavily.com/mcp/",
      "headers": { "X-API-Key": "${TAVILY_API_KEY}" }
    }
  }
}
```

VIDEO SAID → DOCS SAY

Video mein `.claude\.mcp.json` bhi banayi gayi (repo mein woh khaali hai; asli file root par hai). Docs: **.mcp.json project root par rehti hai, .claude ke andar nahi.** Sirf apne liye server → `claude mcp add --scope user ...` (yeh `%USERPROFILE%\.claude.json` mein jaata hai).

VERIFIED

`${TAVILY_API_KEY}` — secret environment se aata hai, file mein nahi. Isi liye .mcp.json commit karna safe hai.

skills\ aur agents\

	skills\<name>\SKILL.md	agents\<name>.md
Kya	Reusable instructions/workflow	Subagent — apna prompt, tools, model
Kaise chalta	`/name` ya Claude khud relevant samjhe	Delegate hone par, **alag context window**
Misaal	`/review-code`, `/run-evals`	security-auditor, test-writer

`commands\*.md` purana tareeqa hai (same mechanism) — naya kaam skills mein karo; name clash par skill jeet-ti hai.

CHAPTER 10

Sabse important concept: Context vs Enforcement

Yeh baat video mein seedhi nahi kahi gayi, lekin docs isay bar bar repeat karti hain — aur AI engineer ke liye yeh sab se zaroori lesson hai.

CONTEXT (suggestion)

CLAUDE.md · CLAUDE.local.md
rules*.md · skills · auto memory

Model parhta hai aur follow karne ki koshish karta hai.

Zyada tar follow — guarantee nahi

ENFORCEMENT (rule)

settings.json permissions
hooks · managed settings · sandbox

Claude Code (program) check karta hai — model nahi.

Har haal mein lagu

Aap chahte ho...	Kahan likho
"Docstrings likho", "camelCase use karo"	rules\code-style.md (context)
"Kabhi rm -rf mat chalao"	settings.json → `deny` (enforced)
".env kabhi mat parho"	settings.json → `Read(./.env)` deny
"Har edit ke baad formatter chalao"	settings.json → hooks
"Main branch par commit mat karo"	CLAUDE.md (guidance) + hook/branch protection (enforcement)

SEEDHI BAAT

CLAUDE.md mein "please delete mat karna" likhna = request. settings.json mein deny likhna = taala. Jo cheez nuksaan kare, uske liye hamesha taala lagao.

ENGINEER TIP — LLM APPS MEIN BHI SAME PATTERN

Aap jo bots/agents banate ho unmein bhi yahi rule: system prompt (context) se behaviour shape karo, lekin dangerous actions (refunds, deletes, DB writes) ko **code-level checks** se guard karo. Prompt kabhi security boundary nahi hota.

CHAPTER 11

AI Engineer ka real .claude setup

Misaal: Python FastAPI par ek LLM support-bot / RAG project. Yeh poora structure copy karke apne project ke mutabiq badlo.

```
C:\Projects\ai-support-bot\
├── CLAUDE.md              # <150 lines: stack, commands, architecture
├── CLAUDE.local.md        # Windows paths, local ports (gitignored)
├── .mcp.json              # e.g. docs/search MCP for the team
├── .env                   # keys — gitignored + Read deny
└── .claude\
    ├── settings.json
    ├── rules\
    │   ├── code-style.md      # PEP8, type hints, docstrings
    │   ├── security.md        # no keys in code, PII masking in logs
    │   ├── prompts.md         # paths: prompts/**, src/llm/**
    │   └── testing.md         # paths: tests/** — pytest, mock LLM calls
    ├── skills\run-evals\SKILL.md
    └── agents\security-auditor.md
```

CLAUDE.md (starter)

```
# AI Support Bot
FastAPI webhook → retrieval → LLM → reply. Python 3.12.
## Commands
- Run:   uvicorn app.main:app --reload --port 9000
- Test:  python -m pytest -q
- Evals: python -m evals.run --suite smoke
## Architecture
- app/routes/   webhook handlers only, no business logic
- app/llm/      all model calls via llm_client.py (retries, timeouts, logging)
- prompts/      versioned prompt files
## Rules
- Never call the LLM SDK directly outside app/llm/
- Any change to prompts/ must be followed by the smoke evals
```

rules\security.md (AI-specific)

```
- Never hardcode API keys; read from environment via settings.py
- Mask phone numbers / emails before logging user messages
- Treat user messages and retrieved docs as untrusted (prompt injection)
- Never put secrets or customer data into prompts sent to third parties
```

GLOBAL VS PROJECT — AI ENGINEER VERSION

Global (`%USERPROFILE%\.claude\`): "main Windows par hoon, PowerShell commands do", aap ki personal style, har project ki common `deny` list (rm -rf, .env read).
Project: us project ka stack, commands, prompts/eval rules, team permissions.
Enterprise: company ke security/compliance rules — jab poori team par lagana ho (naye intern ka code bhi same standard par).

CHAPTER 12

Windows par setup — step by step

Video Mac terminal par tha (`mkdir -p`, `touch`, `ls -la`). Windows PowerShell ke equivalents:

Video (Mac)	Windows PowerShell
`mkdir -p .claude`	`mkdir .claude`
`touch CLAUDE.md`	`New-Item -ItemType File CLAUDE.md`
`ls -la`	`Get-ChildItem -Force`
`cd .claude`	`cd .claude`
`~/.claude`	`$env:USERPROFILE\.claude`

Poora project skeleton — ek baar mein

```
# 1) Project folder mein jao
cd C:\Projects\my-app

# 2) Folders
mkdir .claude\rules, .claude\skills, .claude\agents

# 3) Files (sahi jagah par!)
New-Item -ItemType File CLAUDE.md, CLAUDE.local.md, .mcp.json
New-Item -ItemType File .claude\settings.json
New-Item -ItemType File .claude\rules\code-
style.md, .claude\rules\security.md, .claude\rules\testing.md

# 4) Personal files git se bahar
Add-Content .gitignore "CLAUDE.local.md"
Add-Content .gitignore ".claude/settings.local.json"

# 5) Check
Get-ChildItem -Force -Recurse .claude | Select-Object FullName

# 6) VS Code mein kholo
code .
```

PHIR CLAUDE CODE KE ANDAR

`claude` chalao → `/init` (CLAUDE.md draft) → `/memory` (kaunsi memory files load hain + auto memory on/off) → `/permissions` (rules dekho) → `/status` (settings sources). Test: "delete the utils folder recursively" bolo — deny ka message aana chahiye.

SHORTCUT

Video ki tip: JSON khud mat likho — Claude se kaho "mere project ke liye settings.json banao jo pytest/ git allow kare aur rm -rf, Remove-Item, .env read deny kare". Phir **khud parh kar verify** karo (Ch. 6 ka syntax table).

CHAPTER 13

Video said → Docs say

Video ka concept 90% sahi hai; yeh details official docs (21 Oct 2026) se match nahi karti thi:

Video / repo	Docs (verified)
Dot se folder hidden ho jata hai	Linux/Mac par haan. **Windows par nahi** — hidden ek attribute hai; `.claude` Explorer mein normally dikhta hai
`CLAUDE.local.md` ko `.claude\` mein rakha	Documented location: **project root** `.\CLAUDE.local.md`
Git ".local" files khud ignore karta hai	Nahi. CLAUDE.local.md **aap .gitignore mein daalo**. settings.local.json ko Claude Code tab exclude karta hai jab woh khud file likhe
`.claude\.mcp.json` banayi	`.mcp.json` **project root** par; .claude mein nahi
Rule files mein `description:`	Sirf `paths:` parha jata hai; description ignore
`Bash(git commit*)` (no space)	Chalega, lekin docs ka style `Bash(git commit *)`; space ke baghair zyada match karta hai (jaise `ls*` → `lsof`)
deny = Claude delete nahi kar sakta	Deny us form ko rokta hai jo Claude likhta hai; full protection ke liye hooks/ sandbox
Enterprise rules sab se upar, project sab se strong	Dono sach: **instructions** mein project ko priority; **managed settings** ko koi override nahi kar sakta
Local key ke liye OpenAI key env/ settings mein	Keys settings files mein mat likho — env variable / .env + Read deny
Enterprise folder Mac/Linux style	Windows: `C:\Program Files\ClaudeCode\` (managed-settings.json, CLAUDE.md). Purana `C:\ProgramData\...` ab nahi parha jata

EXTRA CHEEZEN JO DOCS MEIN HAIN (VIDEO MEIN NAHI)

- **Auto memory**: `%USERPROFILE%\.claude\projects\<project>\memory\MEMORY.md` — Claude khud learnings likhta hai; pehli 200 lines/25KB har session load. `/memory` se toggle.
- **AGENTS.md**: agar repo mein CLAUDE.md nahi aur AGENTS.md hai, Claude Code usay parh leta hai.
- **agent-memory**, **output-styles**, **workflows** folders bhi .claude mein ho sakte hain.
- `CLAUDE_CONFIG_DIR` env variable se global .claude ki jagah badal sakte ho.

CHAPTER 14

Revision cheat-sheet

File	Ek line mein	Commit?	Type
`CLAUDE.md`	Project ka briefing doc, <200 lines	✔	Context
`CLAUDE.local.md`	Personal notes, root par	✘	Context
`.claude\settings.json`	Team permissions/env/hooks	✔	Enforced
`.claude\settings.local.json`	Personal overrides	✘	Enforced
`.claude\rules\*.md`	Topic-wise rules, optional `paths`	✔	Context
`.mcp.json`	Team MCP servers, root par	✔	Config
`%USERPROFILE%\.claude\`	Global: har project par	—	Both
`C:\Program Files\ClaudeCode\`	Enterprise managed	—	Both

5 cheezen jo hamesha yaad rakhni hain

- **Closest wins** — chat > project > global (instructions); managed settings untouchable.
- **Context ≠ Enforcement** — khatarnaak cheezon ke liye settings/hooks, CLAUDE.md nahi.
- **Order: deny → ask → allow**; space before `*` matters.
- **Bulky CLAUDE.md = bura** — rules\ aur skills mein baanto.
- **.local = personal = gitignore.**

Practice tasks (apne Windows PC par)

1. Kisi test folder mein Chapter 12 ka skeleton banao. `Get-ChildItem -Force` se dekho.
2. `rules\code-style.md` mein "every Python function needs a docstring and type hints" likho → Claude se ek function banwao → check karo.
3. `settings.json` mein `Bash(rm -rf *)` aur `PowerShell(Remove-Item *)` deny karo → Claude se ek folder delete karwane ki koshish karo → denied message dekho.
4. `Read(./.env)` deny karo, ek dummy .env banao → "read my .env" bolo.
5. Ek path-scoped rule (`paths: ["tests/**"]`) banao → `/memory` aur behaviour se confirm karo ke sirf tests chhoone par load hota hai.
6. Global `%USERPROFILE%\.claude\CLAUDE.md` mein likho "I use Windows + PowerShell; give Windows commands only" — ab har project mein lagega.

NEXT VIDEO (5)

CLAUDE.md in depth — kya likhna hai, kya nahi, aur best-practice structure. Transcript bhejo to uski PDF bhi isi style mein bana dunga.

Sources: code.claude.com/docs/en/claude-directory · /memory · /settings · /permissions · /managed-settings (accessed 21 Oct 2026). Course repo: github.com/mayank953/Youtube → Claude Code Crash Course/ClaudeCode/.claude

CLAUDE CODE CRASH COURSE · VIDEO 5 OF 6 · ROMAN URDU

Mastering CLAUDE.md

Woh ek file jo decide karti hai ke Claude Code aap ke project mein kitna acha kaam karega — yeh kya hai, kaise load hoti hai, is mein kya hona chahiye, aur isay chhota (lean) kaise rakhna hai. Concept-first notes, AI-engineer nazariye se, sirf Windows paths ke saath.

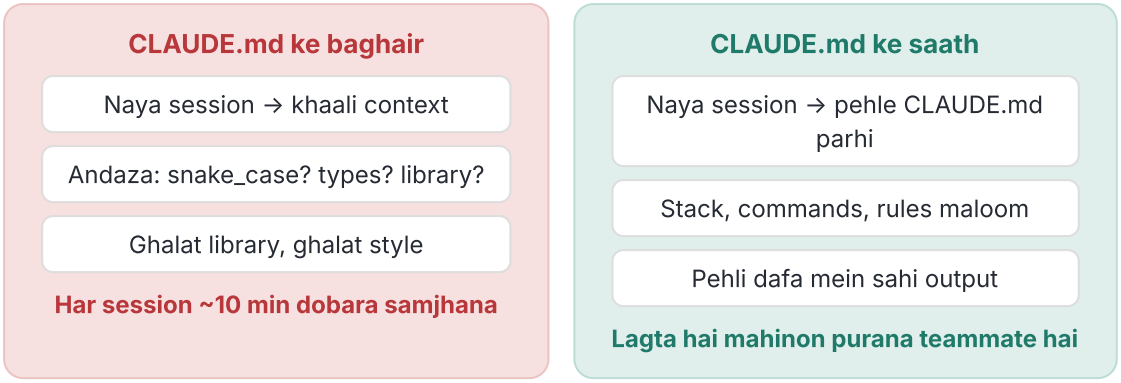
01 Blank-slate wala masla
02 CLAUDE.md asal mein kaise load hoti hai
03 Instruction budget
04 Locations aur hierarchy (Windows)
05 Kya rakhna hai, kya nahi
06 Achhi file ki anatomy
07 Human notes (HTML comments)
08 Primacy aur recency bias
09 Progressive disclosure
10 /init — shuru karo, phir kaato
11 CLAUDE.md vs Hooks
12 Maintenance: isay tech debt samjho
13 AI-engineer playbook
14 Demo recap: Job Tracker
15 Cheat sheet aur self-test

Source: Mayank ka YouTube course (Video 5 transcript) + course repo mayank953/Youtube → Claude Code Crash Course + official Claude Code memory docs (code.claude.com/docs/en/memory). Jo baatein video mein dheeli tarah kahi gayin, unhein docs se check kiya gaya hai aur jahan farq hai wahan flag kiya gaya hai.

01 Blank-slate wala masla

Claude Code har session project ki zero yaad ke saath shuru karta hai. CLAUDE.md woh briefing document hai jo woh kisi bhi file ko chhoone se pehle parhta hai.

Har naya session ek fresh context window hota hai. Claude ko yaad nahi ke kal aap ne kya banaya, aap ki naming conventions kya hain, kaunsi library chuni thi, ya tests kaise chalte hain. Briefing ke baghair woh **andaza lagata hai** — aur har session mein andaza alag hota hai.



Same model, same prompt — alag results. Context andaze khatam kar deta hai.

SIMPLE SAMJHO

CLAUDE.md ek naye team member ko diya gaya **onboarding note** hai. Jitna clear aur short note, utna jaldi woh sahi kaam karega. Note na ho to har din pehle 10 minute project samjhane mein jaate hain.

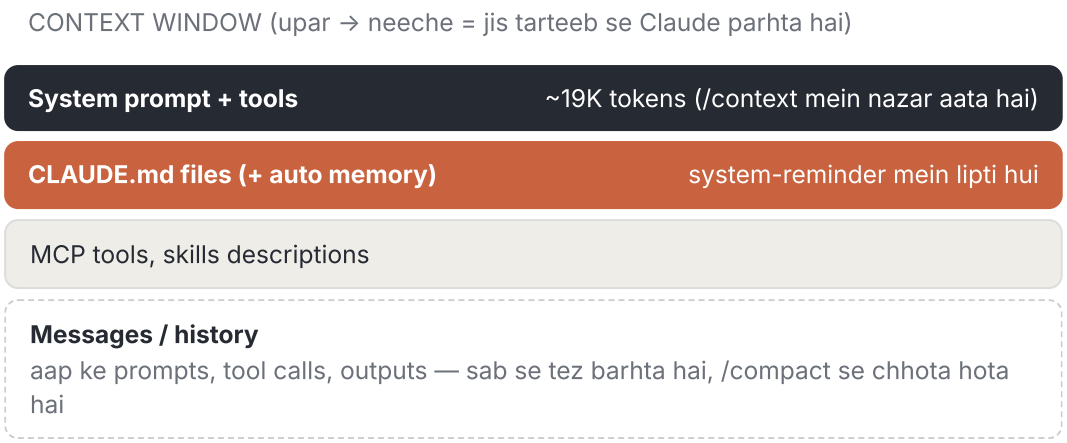
Key definitions

Term	Matlab
CLAUDE.md	Ek markdown file jo *aap* likhte ho. Mustaqil instructions (stack, commands, conventions, rules) jo har session ke shuru mein load hoti hain.
Auto memory	Notes jo *Claude* apne liye likhta hai (seekhi hui baatein, aap ki corrections) `MEMORY.md` mein. CLAUDE.md se alag.
Rules / agent_docs	Extra instruction files jo sirf zaroorat par load hoti hain — "on-demand" layer.

Course notebook se: CLAUDE.md = persistent instructions layer · Auto memory = seekha hua behaviour layer · Rules/agent_docs = on-demand context layer.

02 CLAUDE.md asal mein kaise load hoti hai

Yeh koi config file nahi jiska Claude hukm maane. Yeh conversation mein daala gaya text hai — aur Claude ko bataya jata hai ke woh isay ignore bhi kar sakta hai.



Woh line jis ki koi baat nahi karta

Claude Code aap ki CLAUDE.md ko user message ke andar ek `<system-reminder>` mein lapet deta hai, saath ek note ke jo model ko kehta hai (paraphrase): *yeh context relevant ho bhi sakta hai aur nahi bhi — sirf tab amal karo jab task se bohat zyada relevant ho.*

IS KA MATLAB

CLAUDE.md ko **koi free pass nahi milta**. File mein hona ≠ authority. Agar file current task se ghair-mutalliq cheezon se bhari hai, to Claude ko unhein skip karne ki ijazat hai — aur kaam ki lines bhi shor ke saath skip ho jaati hain. Official docs bhi yahi kehte hain: Claude CLAUDE.md ko **context samajhta hai, enforced configuration nahi**.

Mental model

- **Asar daalti hai, zabardasti nahi karti.** Behaviour ko shape karti hai; guarantee nahi de sakti.
- **Relevance attention jeetti hai.** Har line baaki har line se, aap ke prompt aur history se muqabla karti hai.
- **Kisi cheez ko pakka block karna ho** (jaise `.env` kabhi na chhoona), to hook ya permission deny rule use karo — CLAUDE.md ka ek jumla nahi. (Section 11)

```
# Khud dekho, Claude Code ke andar (PowerShell terminal)
/context      # token ka batwara: system, memory files, tools, messages
/memory       # kaunsi CLAUDE.md / memory files load hain, unhein edit ke liye kholta hai
```

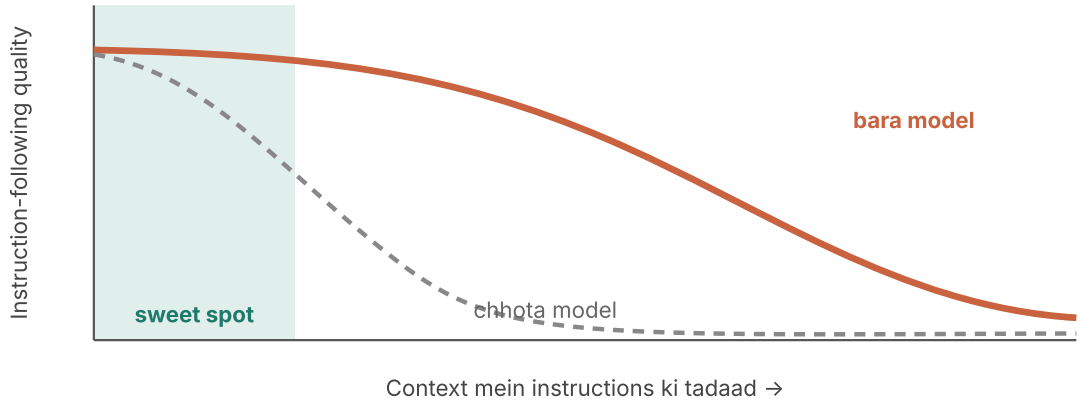
03 Instruction budget

LLMs ek waqt mein mehdood instructions hi follow kar sakte hain. Zyada instructions → *sab* ki quality kam.

Video HumanLayer ke article *"Writing a good CLAUDE.md"* par tika hai. Us ke main daawe:

- Frontier thinking models taqreeban **150–200 instructions** theek consistency se follow karte hain.
- Claude Code ka apna system prompt pehle hi **~50** use kar leta hai.
- To aap ki CLAUDE.md + rules + baaki sab bache hue ~100–150 slots share karte hain.
- Instructions barhne par adherence **har jagah barabar girti hai** — sirf aakhri wali nahi ghum hoti.

- Chhote models (jaise Haiku) bare models se jaldi kharab hote hain.
- Models prompt ke **kinaron** (shuru aur aakhir) wali instructions ki taraf jhukte hain — Section 08 dekho.



Sirf shakal samjhane ke liye, measured data nahi: tadaad barhne par quality girti hai, chhote models mein zyada tezi se.

GOLDEN RATIOS

Relevance > length.
Clarity > quantity.
60 relevant lines 400 shor wali lines se behtar hain.

SIMPLE SAMJHO

Agar meeting mein 300 instructions do, banda 10 bhi theek se yaad nahi rakhta. 20 sharp instructions do, sab follow hoti hain.

Official size guidance

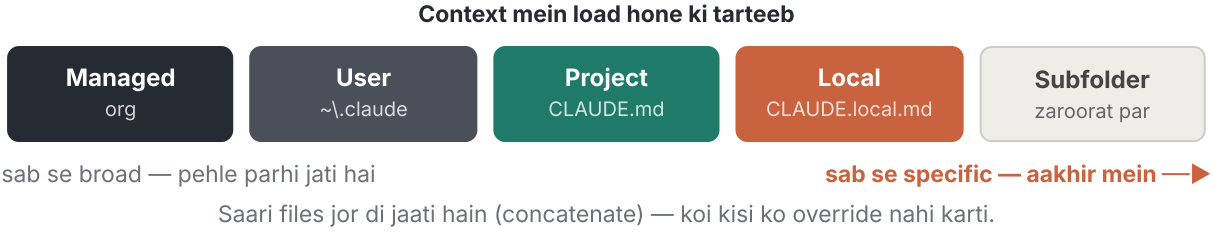
Anthropic ke docs: **har CLAUDE.md 200 lines se kam** rakho. Lambi files zyada context khaati hain aur adherence kam karti hain. Instructions itni concrete likho ke check ho sakein:

Mubham ❌	Check ho sakne wali ✅
Code theek se format karo	2-space indentation use karo
Apni changes test karo	Commit se pehle `npm test` chalao
Files organised rakho	API handlers `src/api/handlers/` mein rehte hain

04 Locations aur hierarchy (Windows)

CLAUDE.md kai levels par reh sakti hai. Aap isay kahan rakhte ho, yeh decide karta hai ke yeh kis par laagu hogi.

Level	Windows location	Kis ke liye	Kis ke saath share
Managed policy (enterprise)	`C:\Program Files\ClaudeCode\CLAUDE.md`	Company standards, security aur compliance. IT set karta hai; exclude nahi ho sakti.	Machine par sab
User (global)	`%USERPROFILE%\.claude\CLAUDE.md` misaal: C:\Users\you\.claude\CLAUDE.md	Aap ki personal preferences, saare projects mein	Sirf aap, saare projects
Project	`.\CLAUDE.md` ya `.\.claude\CLAUDE.md`	Architecture, commands, conventions — git mein commit karo	Poori team
Local	`.\CLAUDE.local.md`	Aap ke private project notes (sandbox URLs, test data). `.gitignore` mein daalo.	Sirf aap, yeh project
Auto memory	`%USERPROFILE%\.claude\projects\<project>\memory\MEMORY.md`	Claude ki apni seekhi hui baatein. Har session pehli **200 lines / 25KB** load hoti hain.	Sirf aap



VIDEO KE MUQABLE MEIN CORRECTION

Video kehta hai "local ki priority sab se zyada hai". Official docs ke mutabiq files **override nahi hoti** — sab **concatenate** hoti hain. Zyada specific files bas **baad mein** aati hain, is liye Claude unhein aakhir mein parhta hai (aur recency bias ki wajah se woh "zyada strong" lagti hain). Agar do files mein **takraao** ho, to Claude kisi ko bhi chun sakta hai — is liye "priority" par bharosa karne ke bajaye levels ko consistent rakho.

Directory walk

- Claude Code launch par aap ke **current folder aur har parent folder** se `CLAUDE.md` + `CLAUDE.local.md` load karta hai. `C:\code\foo\bar` mein chalao → `foo\bar\CLAUDE.md`, `foo\CLAUDE.md` aur saath wali local files load hongi.
- Aap ke neeche **subfolders** wali CLAUDE.md files **zaroorat par** load hoti hain — sirf jab Claude us subfolder ki koi file parhe/edit kare.
- Bara monorepo doosri teams ki files utha raha hai? `.claude\settings.local.json` mein `claudeMdExcludes` use karo.
- Repo pehle se `AGENTS.md` (Codex/Cursor style) use karta hai? Jab CLAUDE.md na ho to Claude usay parh leta hai, ya CLAUDE.md ke andar `@AGENTS.md` likh do. Windows par symlink ke bajaye import behtar hai.

05 Kya rakhna hai, kya nahi

Har line ka test: "Kya isay hatane se Claude zyada tar sessions mein ghalti karega?" Agar nahi — kaat do.

✅ **RAKHNA HAI**

- **Overview** — project kya hai, stack, ek do lines
- **Commands** — dev, test, typecheck, build, lint (sab se qeemti)
- **Naming conventions** — chhoti, essay nahi
- **Folder structure** — sirf ahem folders
- **Testing** — framework + ek test kaise chalana hai
- **Hard rules / guardrails** — main par kabhi direct commit nahi, koi hardcoded secrets nahi
- **Verification** — Claude apna kaam kaise check kare
- Gehre topics ke liye agent_docs / rules ki taraf **pointers**

❌ **NAHI RAKHNA**

- Poori **API documentation** → link/pointer do
- **Code snippets / code dumps** — purane ho jaate hain; asal file ki taraf ishara karo
- **Purani (outdated)** instructions
- **Task-specific** instructions — woh chat mein bolo; CLAUDE.md *project* ke liye hai
- **Lambi style guides** — linter/formatter use karo; "LLM ko linter mat banao"
- **Hook logic** / har dafa chalne wale steps → asal hooks
- Un masloon ke rules jo **pehle hi hal** ho chuke

Commands #1 section kyun hai

Kaam karte hue Claude ko baar baar kuch chala kar **verify** karna hota hai — tests, type checks, build. Agar exact command likhi na ho to woh andaza lagata hai (`npm test`? `pnpm test`? `pytest`? `uv run pytest`?). Har ghalat andaza har session mein tokens aur waqt khaata hai. Anthropic ka apna mashwara: bash commands, code style aur workflow rules shamil karo.

Line kab add karni hai (official triggers)

- Claude **wahi ghalti doosri dafa** kare.
- Code review mein koi aisi cheez pakri jaye jo Claude ko codebase ke baare mein **pata honi chahiye thi**.
- Aap khud ko chat mein wahi correction **dobara** type karte paao.

WHAT / WHY / HOW FRAMEWORK (COURSE NOTEBOOK SE)

WHAT — tech aur structure (Next.js + Prisma; folders). **WHY** — maqsad (ek local job-tracker; ek SaaS dashboard). **HOW** — workflows (`npm run dev` , `npm test` , verify steps). Agar koi line WHAT, WHY ya HOW nahi, to us par sawal uthao.

06 Achhi CLAUDE.md ki anatomy

Video wali haath se likhi file (≈60 lines) — "achhi" reference. Har section chhota aur check ho sakne wala.

```
<!-- HUMAN NOTE: Don't remove legacy auth yet. Migration pending from v1 API. -->
# Project: Job Application Tracker
A Kanban-style web app for tracking job applications.
Stack: Next.js 14, TypeScript, Tailwind CSS, Supabase.
The app runs at localhost:3000 during development.

## Essential Commands
- Start dev server: `npm run dev`
- Run tests: `npm test`
- Type check: `npm run typecheck`
- Build: `npm run build`   - Lint: `npm run lint`

## Conventions
- Components: PascalCase (TaskCard, KanbanColumn)
- Functions/variables: camelCase   - CSS classes: kebab-case
- Files: kebab-case.tsx   - API routes: /api/resource-name (REST)

## Structure
- /app — Next.js app directory   - /components — reusable UI
- /lib — utilities   - /types — TypeScript types
State management: React Context (no Redux)

## Testing
- Framework: Vitest + React Testing Library
- Single test: `npm test -- --grep "TestName"`
- Test files mirror source: Button.tsx → Button.test.tsx

## Rules — Always Follow These
- Never commit directly to main
- Never hardcode credentials — always use environment variables
- Always handle errors explicitly — no silent failures
- Prefer explicit over clever — readable code beats compact code

## Verification
- After code changes: run `npm run typecheck && npm test`
- After UI changes: take a screenshot and compare to requirements
- After database changes: verify with a test query
- When uncertain: ask rather than assume
```

Note: file ka content English mein hi rakha hai — CLAUDE.md asal mein aise hi likhi jaati hai.

Section	Kyun hai
Overview (3 lines)	Ghalat stack ke andaze rokta hai. Port likhne se "kaunsa URL?" wale sawal khatam.
Commands	Claude inhein chala kar verify karta hai — exact strings, koi andaza nahi.
Conventions	Sessions ke darmiyan naming ka farq khatam. Bullets, prose nahi.
Structure	Claude ko explore kiye baghair pata hota hai nayi files kahan jayengi (tokens bachte hain).
Rules	Guardrails. Anthropic ki internal teams: implementation ke baad hamesha tests chalao.
Verification (aakhir mein!)	Jaan boojh kar aakhir mein rakha — recency bias (Section 08).

07 Human notes — HTML comments

Docs mein chhupa hua feature: insaano ke liye notes jin par Claude ka zero token kharch hota hai.

CLAUDE.md mein block-level HTML comments content ko Claude ke context mein daalne se **pehle hata diye jaate hain**. Inhein maintainer notes, team reminders, ya file ko phoolne se bachane wali "yeh mat daalna" list ke liye use karo.

```
<!--
HUMAN NOTE:
- Don't remove legacy auth yet
- Migration pending from v1 API
-->

# Project: ...

<!--
DON'T ADD:
- Entire API documentation
- Outdated code snippets
- Task-specific instructions
- Hook-related instructions
-->
```

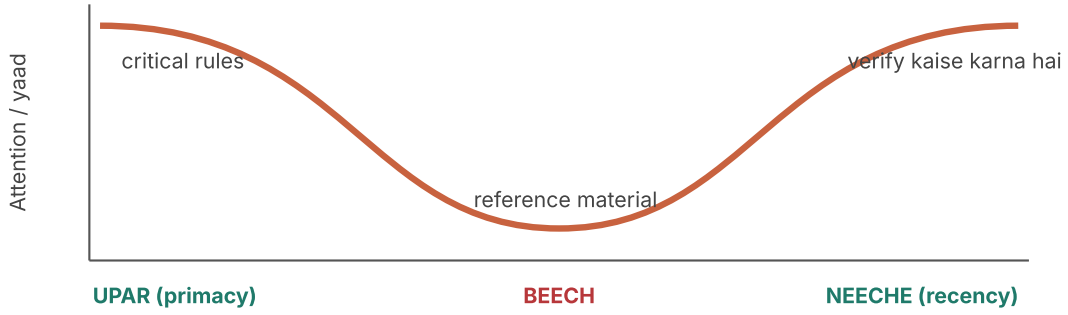
- **Code blocks** ke andar wale comments mehfooz rehte hain (hataye nahi jaate).
- Agar Claude file ko apne Read tool se seedha khole to woh inhein *dekh* sakta hai — yeh auto-inject hone wale context se chhupe hain, secret nahi. Passwords wahan mat rakho.
- Company mein bohat kaam ke: yeh notes repo ke saath chalte hain aur ek hi file se insaano aur AI dono ko onboard karte hain.

SIMPLE SAMJHO

`<!-- -->` ke andar likha = insaano ke liye. Claude ko auto-load mein nahi milta, token bhi nahi lagte. Team ke liye "yeh cheezein is file mein mat daalna" wali list yahan rakho.

08 Primacy aur recency bias

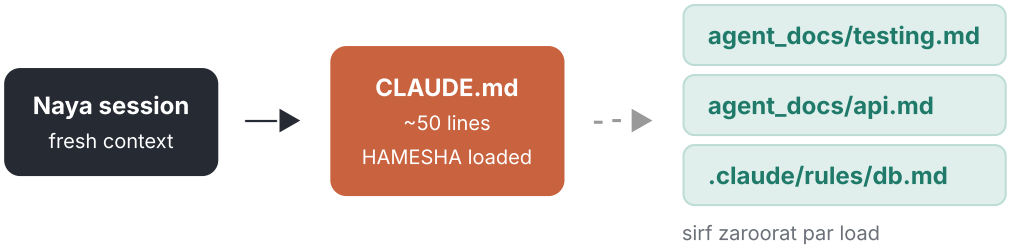
Lambi meeting ki tarah: shuru aur aakhir yaad rehta hai; beech ka hissa dhundla ho jata hai.



Critical guardrails upar rakho, verification bilkul aakhir mein. Misaal: "run tests" beech mein ho to bhool sakta hai; aakhir mein ho to kaam khatam karne se pehle Claude ki parhi hui aakhri cheez hoti hai. Lambi chats mein bhi yahi logic — conversation ke beech mein dabi instructions feeki par jaati hain, yeh `/compact` karne ya fresh shuru karne ki ek wajah hai.

09 Progressive disclosure

CLAUDE.md ko chhota aur hamesha-loaded rakho; gehra context un files mein le jao jo sirf zaroorat par load hon.



Ek 400-line file ke bajaye **ek chhoti file + modular docs** rakho. CLAUDE.md bas ishara karti hai: *"Testing conventions ke liye agent_docs/testing.md parho."*

Split karne ke teen tareeqe — farq samjho

Mechanism	Syntax	Kab load hota hai / context ka kharch
Pointer (plain text)	`See agent_docs/testing.md`	Sirf agar Claude kholne ka faisla kare. Sab se sasta. Kabhi kabhar chahiye wale gehre docs ke liye best.
@import	`@docs/architecture.md`	**Launch** par expand hota hai — file ko organise karta hai lekin context **nahi** bachata. 4 hops tak gehra. Paths import karne wali file ke relative.
Path-scoped rule	`.claude\rules\api.md` + `paths:` frontmatter	Sirf jab Claude matching files par kaam kare. Baghair `paths` wale rules har session load hote hain.
Skill (agli video)	`.claude\skills\...`	Jab invoke ho ya relevant samjha jaye. Multi-step procedures ke liye.

```
# .claude/rules/api.md — sirf API files edit karte waqt load hoti hai
---
paths:
  - "src/api/**/*.py"
---
- Every endpoint returns {"data": ..., "error": ...}
- Validate request bodies with Pydantic models
```

Recommended skeleton — course repo ki `structure.md`

```
# [Project Name]
## ⚡ Critical Rules — Read These First   ← upar: primacy
- Never [X]   - Always [Y]   - Do not [Z]
## Project              Kya hai, stack, maqsad — ek jumla
## Commands             Sab se zyada istemal hone wala section
## Structure            File layout — darmiyani ahmiyat
## Conventions          Naming, patterns — darmiyani ahmiyat
## Reference Documents  agent_docs/ ki taraf pointers — zaroorat par load
## How to Verify        ← bilkul aakhir: recency bias, khatam karne se pehle check
```

10 `/init` — shuruaat hai, jawab nahi

`/init` aap ka codebase scan kar ke CLAUDE.md bana deta hai. Maujooda project ke liye mufeed — lekin zaroorat se zyada likh deta hai.

- Repo ko analyse kar ke build commands, test instructions aur conventions likhta hai jo usay milein.
- Agar CLAUDE.md pehle se ho to overwrite karne ke bajaye improvements suggest karta hai.
- Kuch guides kehti hain "/init kabhi use mat karo, khud likho". Video ka mauqif: **isay draft ke taur par use karo, phir sakhti se kaat chhaant karo.**

DEMO MEIN KYA HUA

Haath se likhi file **~60 lines** ki thi. `/init` wali **~190 lines** ki thi aur us mein tha: architecture overview, state management par essay, ek TypeScript code snippet, debugging tips, performance notes, "known limitations & future improvements"... Is mein se zyada tar code se khud pata chal sakta hai ya zyada tar tasks ke liye ghair-zaroori hai → kaatna chahiye ya `agent_docs/` mein le jana chahiye.

/init file ke liye trim checklist

Generate hua section	Rakho / Shift karo / Kaato
Quick commands	RAKHO — sab se qeemti
Naming conventions, ahem folders	RAKHO — chhota karo
Architecture deep-dive, DB notes	SHIFT → agent_docs/architecture.md
Code snippets (jaise try/catch pattern)	SHIFT → asal file ki taraf ishara karo
Debugging / performance tips	SHIFT ya KAATO
Known limitations / future ideas	KAATO — roadmap hai, instructions nahi
Har file ka poora file tree	KAATO → sirf top-level folders

VIDEO KA PRO MOVE

Official docs page + HumanLayer article kisi bhi AI mein apne repo summary ke saath paste karo, aur kaho ke in rules ke mutabiq CLAUDE.md likhe. Phir har line khud review karo.

11 CLAUDE.md vs Hooks

CLAUDE.md zyada tar follow hoti hai — video ke mutabiq taqreeban 70–80%. Na-qabil-e-samjhota cheezon ke liye "zyada tar" kaafi nahi.

Zaroorat	Kya use karo
Guidance, conventions, preferences	**CLAUDE.md**
Hamesha chalna chahiye (har edit ke baad format, lint, tests)	**Hook** (jaise PostToolUse) — deterministic, har dafa chalta hai
Kabhi nahi hona chahiye (`.env` parhna, `rm` , main par push)	**PreToolUse hook** ya settings mein **permissions.deny**

Hooks agli video mein. Usool: agar ek dafa bhi follow na hona nuqsan de, to isay markdown file ke ek jumle par mat chhoro.

12 Maintenance — CLAUDE.md ko tech debt samjho

Yeh ek dafa likh kar bhool jaane wali file nahi. Video mein bataya gaya industry practice: waqt ke saath isay ghatao, barhao nahi.

SOCH

"Main CLAUDE.md ko technical debt ki tarah treat karta hoon — waqt ke saath kam karo, barhao nahi."

Practice	Kaise
Har mahine sakhti se review	Har line parho. Poocho "kya yeh ab bhi koi asal ghalti rokti hai?"
Har auto-change review karo	Claude kabhi kabhi khud CLAUDE.md edit karta hai aur add karne ka shauqeen hai. Commit se pehle git mein diffs check karo.
Hal shuda masloon ke rules delete karo	Rule is liye add kiya tha ke Claude ghalti X karta tha? Jab X hona band ho jaye (naya model, refactor), rule hata do.
Copy nahi, pointer	Kabhi code ya API docs paste mat karo. File ki taraf ishara karo — file purani nahi hoti, copy ho jaati hai.
Takraao khatam karo	Do mutazad rules → Claude koi bhi ek chun leta hai. User, project, local aur rules files ek saath check karo.
Claude se audit karwao	Naye Claude Code versions mein purani ya takraane wali instructions dhoondne ke liye `/doctor prompt-audit` hai.

BOTTOM LINE

Ek 40-line CLAUDE.md jo sahi aur relevant ho, us 400-line file se ~10× qeemti hai jo aadhi purani aur aadhi ghair-mutalliq ho. Kam hi asal mein zyada hai.

Working-memory trick (course repo mein dekhi)

Final job-tracker CLAUDE.md ek chhote "Working Memory" rule par khatam hoti hai: har ahem kaam ke baad `CHANGELOG.md` mein ek line add karo (`[YYYY-MM-DD] type: description`) aur session ke shuru mein usay parho. Is se CLAUDE.md phoolaye baghair sessions ke darmiyan sasti continuity mil jaati hai.

```
## Working Memory
After completing any significant action, add one line to CHANGELOG.md:
- Format: `[YYYY-MM-DD] type: brief description` (feat, fix, docs, refactor, test)
- Read CHANGELOG.md at session start to understand project state.
```

13 AI-engineer playbook

Yeh sab LLM apps, agents, RAG pipelines aur bots par kaise laagu karna hai — Windows par.

Misaal: LLM support-bot / RAG service ke liye CLAUDE.md (≈45 lines)

```
# Project: AI Support Bot
## ⚡ Critical Rules
- Never hardcode API keys — read from `.env` via settings.py
- Never change prompt files in /prompts without updating its eval cases
- Never call paid LLM APIs in unit tests — use the mock client in tests/fakes.py

## Project
WhatsApp/web support bot answering store FAQs + order status.
Stack: Python 3.12, FastAPI, uv, OpenAI/Anthropic SDK, Chroma (local vector DB).

## Commands (PowerShell)
- Install: `uv sync`
- Run API: `uv run uvicorn app.main:app --reload --port 8000`
- Tests: `uv run pytest -q`    Single: `uv run pytest -k "test_name"`
- Lint/format: `uv run ruff check . --fix; uv run ruff format .`
- Evals: `uv run python -m evals.run --suite smoke`
- Re-index docs: `uv run python -m rag.ingest data\docs`

## Structure
- app/ — FastAPI routes       - agents/ — tool definitions + agent loop
- rag/ — chunking, embeddings, retrieval       - prompts/ — versioned .md prompts
- evals/ — golden Q&A sets     - tests/ — pytest, mirrors app/

## Conventions
- snake_case functions, PascalCase Pydantic models, type hints everywhere
- Prompts live in files, never inline strings
- Every LLM call goes through `llm/client.py` (logging, retries, token count)

## Reference Documents
- RAG design & chunk sizes: agent_docs/rag.md
- Tool schemas for the agent: agent_docs/tools.md

## How to Verify
- After code changes: `uv run pytest -q` and `uv run ruff check .`
- After prompt/RAG changes: run smoke evals and report the score diff
- When unsure about business rules (refunds, delivery): ask, don't assume
```

AI-SPECIFIC LINES JO HONI CHAHIYEN

- Prompts kahan rehte hain + "prompts ke saath evals update karo"
- "Tests mein asli LLM calls nahi" (paise bachte hain)
- Ek hi LLM client wrapper
- Exact eval command — AI kaam ke liye Claude ka "verify"
- Model names / env var names (keys nahi!)

AI-SPECIFIC CHEEZEIN JO BAHAR RAKHNI HAIN

- Poora prompt text (/prompts ki taraf ishara karo)
- Provider API reference docs
- Embedding experiment logs
- Sample chat transcripts / datasets
- Aaj ka bug ("... mein hallucination theek karo")

Aap ka Windows setup — kya kahan rakhna hai

File	Is mein kya daalo
`C:\Users\you\.claude\CLAUDE.md`	Personal, saare projects: "Main Windows par hoon — PowerShell commands do", "pip ke bajaye uv", "trade-offs mukhtasar samjhao".
`project\CLAUDE.md` (git)	Upar wali team file.
`project\CLAUDE.local.md` (gitignored)	Aap ka test phone number, local ngrok URL, sandbox store ka naam.
`project\.claude\rules\*.md`	Path-scoped rules: `rag/**`, `agents/**`, `prompts/**`.

```
# PowerShell — jaldi setup
New-Item -ItemType Directory -Force "$env:USERPROFILE\.claude" | Out-Null
notepad "$env:USERPROFILE\.claude\CLAUDE.md"     # global file
New-Item CLAUDE.local.md; Add-Content .gitignore "CLAUDE.local.md"
New-Item -ItemType Directory -Force .claude\rules, agent_docs | Out-Null
```

Tip: CLAUDE.md mein kisi path ko import kiye baghair sirf zikr karna ho to usay backticks mein lapeto (`` `@README` ``). @import mein spaces wale path mein har space se pehle backslash lagao.

14 Demo recap — Job Tracker mini-project

Ek CLAUDE.md aur khaali folder se chal kar ek chalti hui Kanban app tak.

#	Step	Sabaq
1	CLAUDE.md haath se likhi (pehle .md lagana bhool gaye)	Filename bilkul `CLAUDE.md` hona chahiye warna Claude auto-load nahi karega.
2	Plan mode, Sonnet + high effort: "create a plan for my project"	Claude ne `git log`, `git status` chalaye, files list kin, phir sirf CLAUDE.md se 8–9 phases plan kiye.
3	"Sab kuch local rakho" → Supabase ki jagah Prisma + SQLite	Execution se pehle plan ko mor do; baad mein theek karne se sasta hai.
4	Plan execute karne ke liye Haiku par switch	Strong model se plan, saste model se simple steps execute. (Baad mein unhein laga Sonnet zyada smooth hota.)
5	Bohat se bash prompts approve kiye (npm install, prisma, mkdir...)	Hamesha "yes" click karne ke bajaye `.claude\settings.json` permissions mein safe commands pehle se approve karo.
6	"48% context left until auto-compact"	Lambe builds context jaldi bharte hain; messages context ka ~29% ban gaye.
7	`npm run dev` → localhost:3000 chal gaya; prompt se drag-and-drop add kiya	Mazboot briefing ke upar vibe coding kaam karti hai.
8	`/compact` → ~101K tokens free hue	Compaction zyada tar messages chhote karta hai. Detail agli video mein.
9	Purani file ka naam badla, "scratch se" `/init` chalaya	~190-line file mili → saboot ke /init ko trim chahiye.

DEMO SE EXTRA TIPS

- Apna plan file mein save karo (jaise PLAN.md) — kabhi kabhi khud save nahi hota, aur naya session usay kho deta hai.
- Doston ne kharab context management se 15–20 din mein $400–500 ka company budget uda diya — agli video ka topic.
- Jab unhon ne "yes, don't ask again" chuna to Claude ne khud `.claude\settings.local.json` bana di.

15 Cheat sheet aur self-test

Agli CLAUDE.md likhne se pehle dohrane ke liye ek page.

10 usool

1. ~200 lines se kam; 40–80 ka target.
2. Har line ek asal, baar baar hone wali ghalti roke.
3. Commands section = exact, copy-paste ho sakne wali strings.
4. Critical rules upar, verification neeche.
5. Bullets aur headers, paragraphs nahi.
6. Concrete aur check ho sakne wala, mubham nahi.
7. Docs/code ki taraf ishara karo; kabhi paste mat karo.
8. Gehre topics → agent_docs / path-scoped rules.
9. Hamesha-karo / kabhi-mat-karo → hooks aur permissions.
10. Har mahine review; add se zyada delete karo.

Yaad rakhne wali commands

Command	Kya karti hai
`/init`	Draft CLAUDE.md banati hai
`/memory`	Loaded memory files dekho/edit karo, auto memory on/off
`/context`	Context ka token breakdown
`/compact`	History ka khulasa kar ke tokens free
`/model`	Opus / Sonnet / Haiku switch
`@path`	CLAUDE.md ke andar file import
`<!-- -->`	Sirf insaano ke liye note, hata diya jata hai

Self-test (jawab har sawal ke neeche)

1. **Claude aap ki CLAUDE.md ki koi line kyun ignore kar sakta hai?**
   Yeh context ke taur par ek note ke saath daali jaati hai ke shayad relevant na ho; bohat zyada ya ghair-mutalliq instructions attention ko patla kar dete hain aur adherence kam hoti hai.
2. **Project aur local CLAUDE.md mein takraao hai. Kaun jeetega?**
   Koi "override" nahi karti — dono concatenate hoti hain; local baad mein parhi jaati hai. Asal takraao mein Claude koi bhi chun sakta hai, is liye takraao khatam karo.
3. **Windows par aap ki global file kahan hoti hai?**
   `%USERPROFILE%\.claude\CLAUDE.md` *(jaise* `C:\Users\you\.claude\CLAUDE.md`*).*
4. **Kya paanch docs @import karne se context bachta hai?**
   Nahi — imports launch par expand hote hain. Zaroorat par load ke liye plain pointers ya path-scoped rules use karo.
5. **"Har edit ke baad hamesha tests chalao" kabhi skip nahi hona chahiye. Yeh kahan jayega?**
   Hook mein. CLAUDE.md zyada tar follow hoti hai, hamesha nahi.
6. **Kaunsa section aakhir mein jata hai aur kyun?**
   How to Verify — recency bias ki wajah se aakhri hissa woh hai jo Claude kaam khatam karne se pehle parhta hai.
7. **"200 lines" wala figure kis par laagu hota hai?**
   Do cheezon par: har CLAUDE.md ki recommended hadd, aur auto memory ki MEMORY.md ki hard load limit (pehli 200 lines ya 25KB).

Resources

- Official docs — How Claude remembers your project: code.claude.com/docs/en/memory
- HumanLayer — "Writing a good CLAUDE.md": humanlayer.dev/blog/writing-a-good-claude-md

- Course repo: github.com/mayank953/Youtube → Claude Code Crash Course → ClaudeCode/CLAUDE-MD (CLAUDE.first.md, CLAUDE/structure.md, /init version) aur SplitMoney/CLAUDE_md_best_practices.ipynb

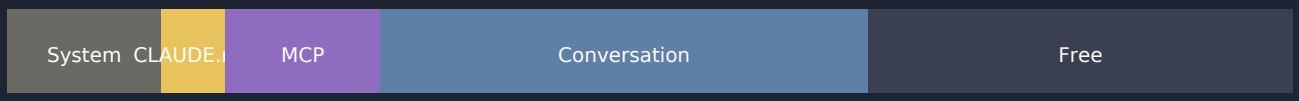
AGLI VIDEO

Context management — context rot, token budget, `/clear` vs `/compact`, aur lambe sessions ko sasta aur sharp kaise rakhna hai.

CLAUDE CODE CRASH COURSE · VIDEO 6

Context Management & Context Rot

Context window kyun bharti hai, rot kaise pehchanein,
aur tokens + paisa kaise bachayein



Topics: context window · statelessness · /context · /compact vs /clear · /btw · status line
CHANGELOG.md technique · specific prompts · token budgeting (6 rules) · AI-engineer use cases

01 Yeh topic itna important kyun hai?

Companies Claude Code ko **per token** pay karti hain (API pricing). Video mein example: kai companies ne poore saal ka AI budget 4–5 mahine mein khatam kar diya. Context samjhe baghair kaam karne wale log same kaam ke liye **5x–7x** zyada kharch kar dete hain.

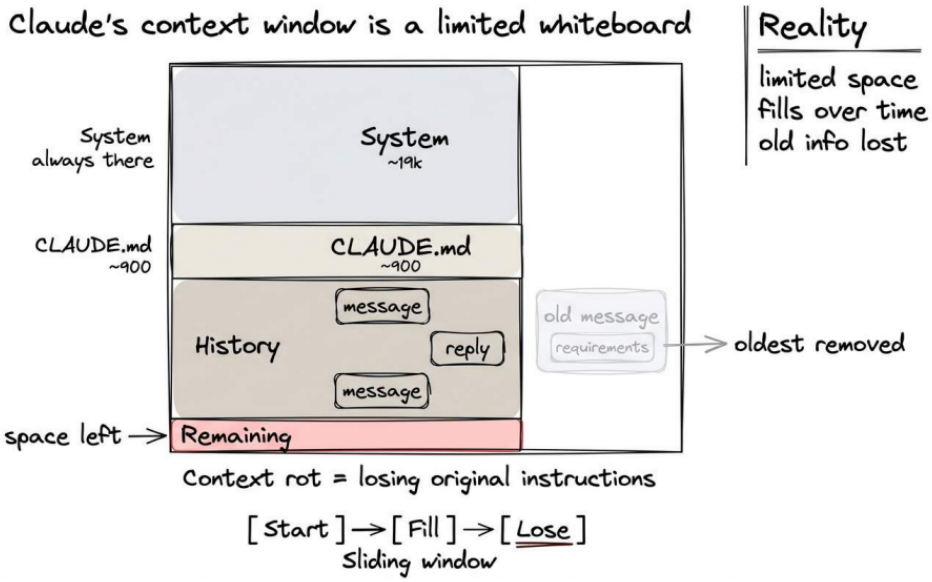
Do problems ek saath hoti hain jab context bhar jaati hai:

- **Quality girti hai** — Claude instructions bhoolne lagta hai, ghaltiyan karta hai (isko **context rot** kehte hain).
- **Cost barhti hai** — har naye message ke saath poori history dobara bheji jaati hai, to har message pichle se mehenga hota hai.

> **Asaan alfaaz mein**
>
> Pro/Max subscription pe aapko seedha bill nazar nahi aata, lekin limit jaldi khatam hoti hai. Company mein API key pe kaam karoge to har faltu token = paisa. Isliye yeh skill junior dev se CTO tak sab ke kaam ki hai.

02 Context window — ek limited whiteboard

Context window woh saari cheezein hain jo Claude ek waqt mein "dekh" sakta hai. Socho ek whiteboard hai jis pe jagah limited hai. Kuch hissa pehle se bhara hota hai (system prompt, tools), phir CLAUDE.md, phir aapki conversation. Jab jagah kam hoti hai to **purani cheezein mit-ti hain** ya compact ho jaati hain — aur aksar wohi purani requirements hoti hain jo aapne shuru mein di thin.



Course diagram: Start → Fill → Lose (sliding window). Context rot = original instructions kho dena.

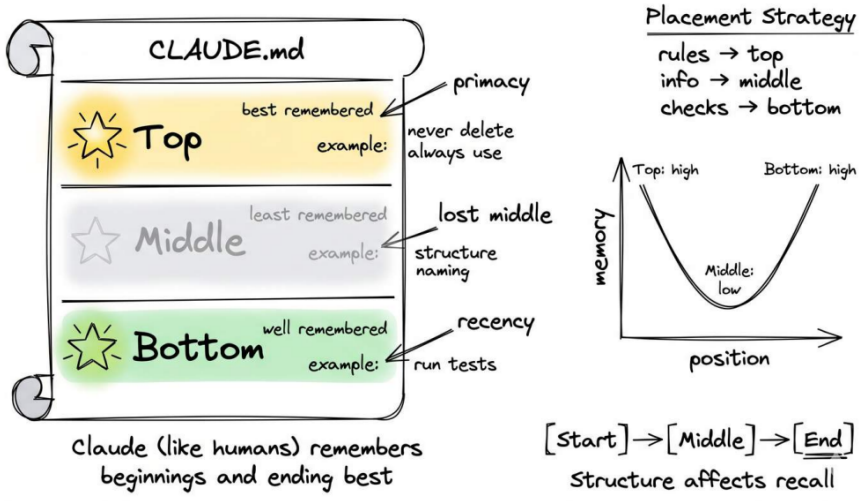
Session start hote hi kya load hota hai (official "Explore the context window" simulation ke example numbers):

Cheez	~Tokens	Aapke control mein?
System prompt	4,200	Nahi — hamesha load, aapko dikhta bhi nahi
Auto memory (MEMORY.md)	680	Thoda — pehli 200 lines / 25KB load hoti hain
Environment info (folder, OS, git)	280	Nahi
MCP tool names (deferred)	120+	**Haan** — jitne zyada servers, utna zyada
Skill descriptions (sirf 1-line)	450	Haan — full skill tabhi load jab use ho
~/.claude/CLAUDE.md (global)	320	**Haan** — lean rakho
Project CLAUDE.md	1,800	**Haan** — lean rakho

Is ke baad har cheez add hoti rehti hai: aapka prompt (~45 tokens), **har file jo Claude parhta hai** (ek file 1–2.5k tokens), grep results, test output, Claude ke replies, hooks ka output. Simulation ki main tip: **file reads sab se zyada context khaati hain.**

> **Try karo**
>
> Official interactive page khud chala ke dekho: code.claude.com/docs/en/context-window — Start session → Send dabate jao aur dekho tokens kaise bharte hain.

Model ki context limits (video ke mutabiq): Haiku ~200k, Sonnet ~200k, Opus / web pe 1M tak. Lekin badi window ka matlab yeh nahi ke problem khatam — **primacy & recency bias** ki wajah se model shuru aur aakhir ki cheezein behtar yaad rakhta hai, beech ki cheezein "lost in the middle" ho jaati hain. 190k tokens bhar ke bhi aap expect nahi kar sakte ke har instruction yaad rahegi.



Primacy and Recency Bias

Course diagram (Video 5 recap): Top aur Bottom best yaad rehte hain, Middle sab se kam.

03 Sab se important concept: LLM calls stateless hain

Claude ko pichle messages **yaad nahi** hote. Har baar jab aap naya message bhejte ho, Claude Code **poori history + system prompt + CLAUDE.md + tools** dobara bhejta hai. Model har call pe "naya baby" hai jo sirf wohi jaanta hai jo us call mein bheja gaya.

Is ka natija: **snowball effect**. Message #20 apne saath pichle 19 messages ka bojh le ke jaata hai, aur aap input tokens ke paise dete ho. Compact karne se yeh bojh kam ho jaata hai:

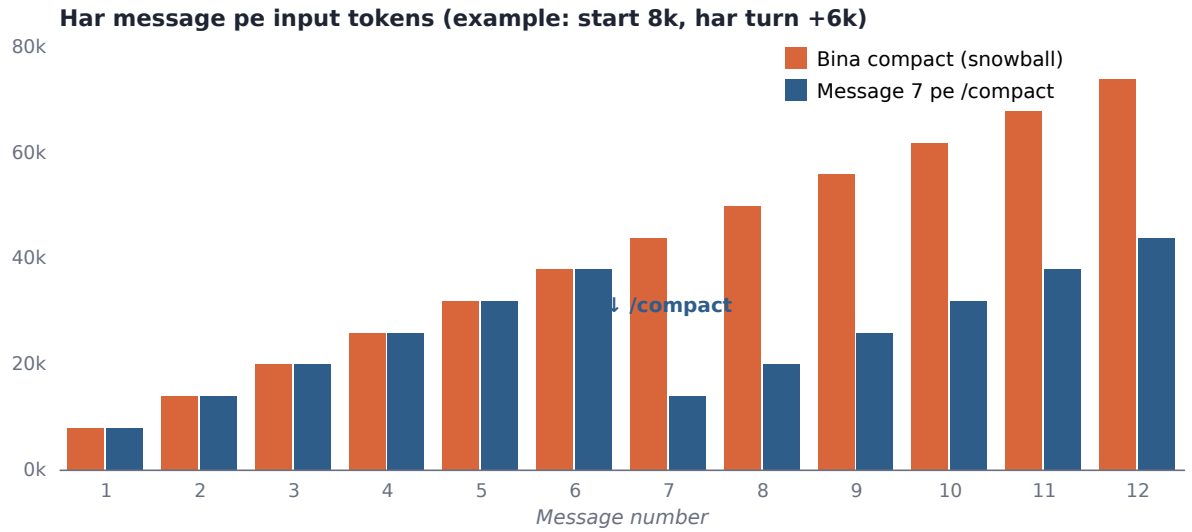


Illustration (round numbers): bina compact ke har turn mehenga hota jaata hai; compact ke baad history ek chhoti summary ban jaati hai.

Asaan alfaaz mein

ChatGPT pe bhi yahi hota hai — lambi chat mein woh camelCase wali ya "formal rakhna" wali baat bhool jaata hai. Claude Code mein yeh zyada hota hai kyun ke woh files parh parh ke context bharta hai. Claude Code = brain (Claude) + hands (tools), to brain ki problems hands ke saath bhi aati hain.

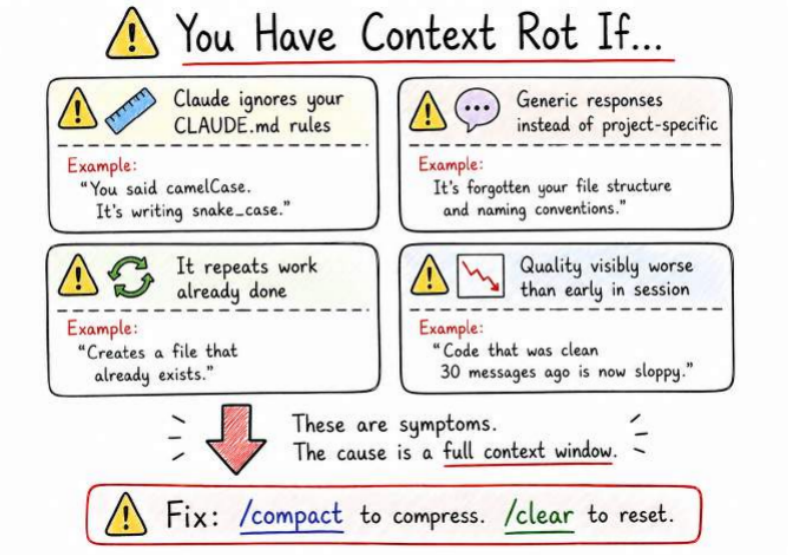
AI Engineer angle — apne projects mein kaise use karein

Apne WhatsApp bot ya kisi bhi chatbot mein bhi aap ko **khud** poori history har API call mein bhejni padti hai — Messages API bhi stateless hai. Lambi customer chats = barhta bill.

Solutions jo aap khud implement karte ho: **sliding window** (last N messages), **running summary** (purane messages ko summarize karke ek message bana do — yahi /compact karta hai), aur **prompt caching** (fixed system prompt / catalog ko cache karo taake baar baar full price na lage).

Har request ke tokens log karo (response mein usage aata hai) — taake pata chale kaunsi conversation mehengi ja rahi hai.

04 Context rot ko kaise pehchanein



Course diagram: yeh symptoms hain — asal wajah full context window hai.

Symptom	Example
CLAUDE.md rules ignore karna	Aapne camelCase kaha tha, woh snake_case likh raha hai
Generic jawab, project-specific nahi	Aapka folder structure / naming convention bhool gaya
Kiya hua kaam dobara karna	Woh file bana raha hai jo pehle se maujood hai
Quality visibly kharab	30 messages pehle saaf code likh raha tha, ab sloppy + errors

Note: Claude Code mein **auto-compact** default on hota hai (config mein `autoCompact: true`) — jab context limit ke qareeb pohanchta hai to khud compact kar deta hai. Lekin auto-compact ka intezaar karne se behtar hai ke aap khud sahi waqt pe decide karo.

05 Context dekhne ke tools

/context — abhi kitne tokens use ho rahe hain, aur kis cheez mein (system, MCP tools, memory files, messages, free space). Video mein: 61.4k → dark mode wala kaam karwaya → 128.7k (64%).

/status aur **/usage** — model, account, aur Pro/Max pe session (5-hour) aur weekly limit ka percentage. API key pe ho to **/cost** se dollar cost dikhti hai.

Status line — terminal ke neeche hamesha dikhne wali line. Video mein Mayank ne Claude se hi likhwaya:

```
update my status line to show a loading bar of how many tokens I have used
out of my total context window, and the current model name.
Also add the default thinking mode and permission mode.
```

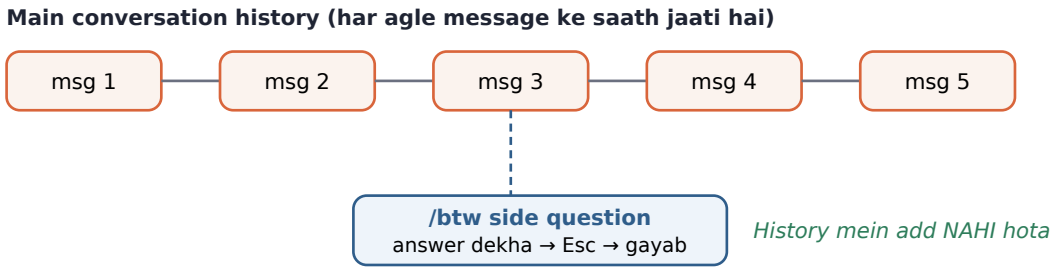
- Claude ne built-in **statusline-setup** skill/agent use ki aur setting `~/.claude/settings.json` (Windows pe: `C:\Users\<aap>\.claude\settings.json`) mein `statusLine` add ki. Yeh **global** setting hai, har project pe lagegi.
- Pehli koshish mein JSON/escaping error aaya, Claude ne khud fix kiya. Change dekhne ke liye Claude Code **band karke dobara kholna** pada.
- Shortcut: seedha `/statusline` likh ke batao kya dikhana hai.

Kyun useful hai

Context bar hamesha saamne ho to aapko pata hota hai kab compact karna hai — /context baar baar chalane ki zaroorat nahi. Lekin status line set karte waqt bhi tokens lage (Mayank ne mazaq mein kaha "lots of tokens are going there") — set karo ek baar aur chhor do.

06 /btw — side question bina history kharab kiye

Lamba kaam chal raha hai aur beech mein ek chhota sa sawaal yaad aaya? `/btw` ("by the way") use karo. Jawab overlay mein aata hai, scroll kar sakte ho, **Esc** dabao aur woh gayab — main conversation history mein add nahi hota.

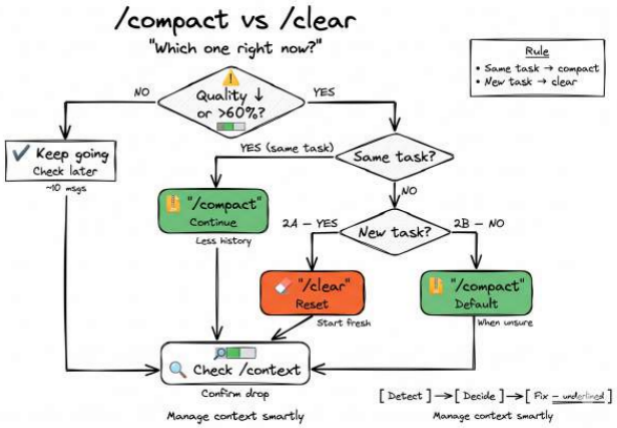



```
/btw what are the latest Claude models and their context windows?
```

Command	Kya karta hai	Kab
/btw	Side question, history mein nahi jaata	Chhota sawaal, kaam disturb nahi karna
/branch (/fork)	Conversation ki copy bana deta hai	Doosra approach try karna, original safe rakhna
Naya session	Bilkul fresh context	Alag task

07 /compact vs /clear

	/compact	/clear
Kya hota hai	Purani conversation ko ek **structured summary** mein badal deta hai (intent, technical goals, decisions)	**Naya session** khali context ke saath. Purana session save rehta hai — `/resume` se wapas aa sakte ho
Kya bachta hai	Startup content (system, CLAUDE.md, memory) dobara load hota hai + summary. Skill descriptions dobara load nahi hoti	Sirf startup content
Nuksaan	Summary mein kuch chhoti details zaroor gum hongi (50k → 5k mein sab nahi aa sakta)	Saari conversation ki baatein naye session mein nahi
Kab	Same task jaari hai, lekin context bhari ho gayi	Naya / bilkul alag task



Course flowchart: Quality ↓ ya >60%? → Same task → /compact · New task → /clear · Pata nahi → /compact (default).

Rule of thumb (video se): ~10 messages ke baad ya context >60% ho to /context check karo. Compact pe focus instruction bhi de sakte ho:

```
/compact focus only on the auth bug we were fixing
```

Video mein: /context 128.7k (64%) tha → /clear se naya session → /resume se purane pe wapas → /compact → **36.3k**. Yaani compact ne ~70% context bacha liya.

Asaan alfaaz mein

Purane versions mein /clear sab kuch mita deta tha. Ab woh naya session shuru karta hai aur purana /resume mein mil jaata hai — to /clear se darne ki zaroorat nahi.

08 CHANGELOG.md technique — sessions ke darmiyan yaaddasht

Problem: purana session itna bhar gaya ke kharab chal raha hai, lekin naya session shuru karo to Claude ko pata hi nahi project kahan tak pohancha. Solution: Claude se khud ek **CHANGELOG.md** maintain karwao, aur CLAUDE.md mein likh do ke har session ke shuru mein usay parhe.

CLAUDE.md ke aakhir mein yeh add karo:

```
## Changelog rule
After completing any significant action, update CHANGELOG.md:
- Add what was built or changed
- Add any decision made that affects future work
- Add any known bugs or incomplete items
Format: STRICTLY one line per item. Keep it short, never pad it.
Read CHANGELOG.md at the start of every session to understand
the current project state.
```

- Video mein pehli dafa Claude ne lambi entries likh din — Mayank ko alag se kehna pada: "*just add single line changes in CHANGELOG.md, update the same in CLAUDE.md*". Isliye **"strictly one line per item"** shuru se hi likho.
- Chhote changes (jaise ek title badalna) pe Claude changelog update nahi karta — yeh theek hai, sirf **significant** kaam record hota hai.
- Naye session mein pehla prompt: `Read CLAUDE.md and CHANGELOG.md, then ...` — agar Claude khud na parhe.

Compact se behtar kyun?

Compact summary Claude khud banata hai aur kuch details gira deta hai. CHANGELOG.md aapki **file** hai — disk pe rehti hai, kabhi compact nahi hoti, git mein commit hoti hai, aur team ke doosre log bhi parh sakte hain.

AI Engineer angle — apne projects mein kaise use karein

WhatsApp bot jaise multi-phase project mein (blueprint phases) har phase ke baad changelog mein ek line: "*Phase 3: order-status webhook done; known bug: COD orders missing tracking*". Naya session 2 lines parh ke seedha kaam pe.

Same pattern apne agents mein: agent ko "notes file" do jo woh har task ke baad update kare — long-running agents ke liye yeh memory ka sab se sasta tareeqa hai.

09 Specific prompts = kam files = kam tokens

Jab prompt vague hota hai ("improve this codebase", "add tests", "make it faster") to Claude **poora codebase scan** karta hai — dozens files parhta hai — aur har file context mein jaati hai. Jab aap exact jagah batate ho to woh sirf 1–2 files parhta hai.

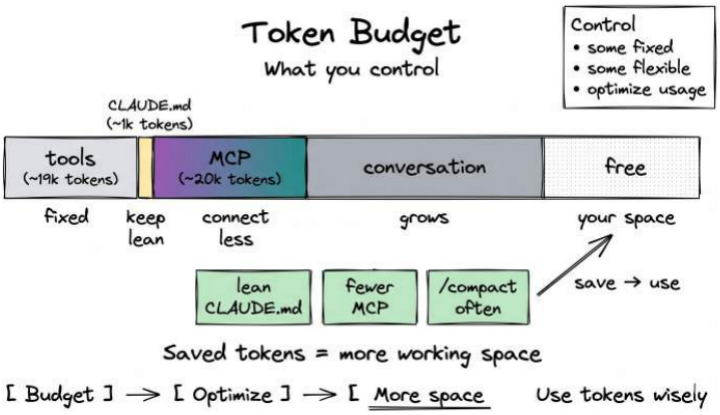
✗ Broad	✓ Specific
Improve this codebase	Fix the null check in auth.js line 47
Add error handling	Add try/catch to the fetchUsers function in api.js
Make the UI better	The button in TaskCard.jsx isn't accessible — add aria-label and keyboard support
Fix the bugs	Delete button throws 'cannot read property of null' — issue is in removeTask() when the list is empty
Add tests	Write Jest tests for addTask() and removeTask() in app.js

VS Code extension trick: code ki lines select karo (video mein ~50 lines Kanban render function ki) aur phir prompt likho — selection Claude ko context ke taur pe milta hai. Result: Claude ne sirf 2 files parhin, pehle chhote se change ke liye bhi bohot saari parh raha tha.

> **Asaan alfaaz mein**
>
> Apne engineer ko bhi aap "kuch theek kar do" nahi kehte — exact batate ho. Claude ke saath bhi yahi. Mayank ke ek dost ka $400–500 ka monthly Claude budget 15–20 tareekh tak khatam ho jaata tha — management ki nazar mein aa jaata hai. Tech samjho, pinpoint karo.

Doosri tip: research-heavy kaam subagent ko do. Official simulation mein subagent ne 6,100 tokens ki files parhin, aur main context mein sirf 420 tokens ka summary aaya. (Subagents Video 7+ mein detail mein.)

10 Token budgeting — 6 rules



Course diagram: kuch hissa fixed (tools), kuch flexible (CLAUDE.md, MCP, conversation). Bachaye tokens = zyada working space.

1. CLAUDE.md lean rakho. Yeh har message ke saath load hoti hai. Video 5 wali baat: monthly review, aggressively prune, solved problems ki instructions hata do, copies ki jagah pointers.

2. Instructions CLAUDE.md se skills / agents / rules mein le jao. CLAUDE.md hamesha load hoti hai; skills ki sirf 1-line description load hoti hai aur full content tabhi jab zaroorat ho. `.claude/rules/` wali files `paths:` pattern se sirf tab load hoti hain jab Claude matching file chhuta hai.

3. MCP server overhead kam karo. Video mein /context ne dikhaya ~14k tokens sirf MCP tools pe ja rahe the, jab ke koi MCP use hi nahi ho raha tha. `/mcp` → har unused server **disable** → /compact → MCP sirf **812 tokens**. Jo use nahi kar rahe, connect mat rakho.

```
/mcp        # list servers, select one -> Disable
/context    # confirm the drop
```

4. Extended thinking context ke liye achhi hai. Thinking ke tokens output ke taur pe bill hote hain, lekin pichle turns ke thinking blocks conversation history mein jama nahi hote — Claude API purane thinking blocks context se nikal deti hai. Bina thinking ke saari reasoning normal reply text ban ke history mein rehti hai. Achhi thinking se kaam kam steps mein bhi hota hai. (Effort/thinking level Video 4/5 mein dekha tha.)

5. Build se pehle plan karo. Plan mode (Shift+Tab) mein pehle approach tay karo. Ghalat direction mein 20 messages = 20 messages ka barhta bill + rot.

6. Code check karo aur versions mein divide karo (V1 review). Pehli iteration ke baad skeleton khud review karo — faltu files, faltu functions, ghalat pattern. V1 ki ghalti V2, V3 mein propagate hoti hai (technical debt). Ghalti mile to alag session mein theek karwao, phir main session pe wapas aao.

> **Bottom line**
>
> Skills, hooks, MCP, agents — aage jo bhi seekhoge, woh mostly **on-demand** load hote hain. Isliye CLAUDE.md mein sab thoosne ke bajaye cheezein sahi jagah rakhna hi asal token budgeting hai.

11 AI Engineer ke liye — yeh concepts apne projects mein

Yeh sirf Claude Code ki baat nahi — jo bhi LLM app aap banate ho (Claude, GPT, Kimi, koi bhi), wahi physics lagti hai. Jo Claude Code andar kar raha hai, woh aapko apne app mein khud karna padta hai:

Claude Code feature	Apne app mein equivalent	Example
Stateless calls, history resend	Messages array khud maintain karna	Bot har customer msg pe pichle msgs bhejta hai
/compact	Summarization memory: purane N msgs → 1 summary msg	20 msgs ke baad cheap model (Haiku) se summary
/clear	Topic change pe naya thread / history reset	Order complete → naya conversation
CLAUDE.md (always loaded)	System prompt — chhota rakho	Store policies sirf zaroori, baqi retrieve karo
Skills / rules (on demand)	RAG / tool calls — data tabhi lao jab chahiye	Product catalog poora mat bhejo, search tool do
MCP overhead	Tool definitions bhi tokens hain	15 tools ki jagah sirf us flow ke tools bhejo
CHANGELOG.md	Persistent memory: DB mein user facts / state	Customer ka naam, last order, preference
Subagents	Sub-calls jo sirf result return karein	Alag call se order lookup, sirf summary wapas
/context, status line	Token logging + dashboards	Har call ka usage.input_tokens log karo

Cost formula yaad rakho: cost ≈ (input tokens × input price) + (output tokens × output price). Lambi chats mein input tokens hi asal kharcha hain kyun ke woh har turn dobara jaate hain. Prompt caching repeated prefix (system prompt, tools) ko sasta kar deti hai — isliye fixed cheezein **shuru mein** rakho aur badalti cheezein baad mein.

Interview / industry line: "I treat context like a budget: lean always-loaded instructions, on-demand retrieval, periodic summarization, and per-request token logging."

12 Quick revision cheat sheet

Command / cheez	Kaam
/context	Context ka breakdown — kya kitni jagah le raha hai
/compact [focus]	History ko summary bana do, same task jaari rakho
/clear	Naya khali session (purana /resume se milta hai)
/resume	Purana session wapas kholo
/btw	Side question, history mein nahi jaata, Esc se band
/status · /usage · /cost	Model/account · plan limits % · API pe dollar cost
/statusline	Terminal ke neeche model + context bar set karo
/mcp	MCP servers dekho, unused disable karo
Shift+Tab	Modes cycle — plan mode pehle
CHANGELOG.md	One-line entries; har session ke shuru mein parho

Golden rules

- Har message poori history ke saath jaata hai → lambi chat = mehenga + bhulakkar.
- ~10 messages ya >60% → /context dekho. Same task → /compact. Naya task → /clear.
- Specific prompt do, file/line batao, code select karo.
- CLAUDE.md chhota, baqi skills/rules mein. Unused MCP band.
- Plan pehle, V1 review karo, ghalti propagate na hone do.
- Compact bhi details khota hai — important decisions CHANGELOG.md mein likhwao.

Official link: code.claude.com/docs/en/context-window (interactive simulation) · Course repo: github.com/mayank953/Youtube → "Claude Code Crash Course" (mind-map PDF jis se diagrams liye gaye) · Agla video: **Skills**.